

PrimeCell™ Dynamic Memory Controller (PL340)

Revision: r0p0

Technical Reference Manual



PrimeCell Dynamic Memory Controller (PL340)

Technical Reference Manual

Copyright © 2004, 2005 ARM Limited. All rights reserved.

Release Information

Change history			
Date	Issue	Confidentiality	Change
22 June 2004	A	Non-Confidential	First release for r0p0.
31 August 2004	B	Non-Confidential	Second release for r0p0.
25 August 2005	C	Non-Confidential	Incorporate erratum. Add information to <i>Exclusive access monitor</i> on page 2-16.

Proprietary Notice

Words and logos marked with ® or ™ are registered trademarks or trademarks owned by ARM Limited, except as otherwise stated below in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by ARM and the party that ARM delivered this document to.

Product Status

The information in this document is final, that is for a developed product.

Web Address

<http://www.arm.com>

Contents

PrimeCell Dynamic Memory Controller (PL340)

Technical Reference Manual

Preface

About this manual	x
Feedback	xiv

Chapter 1

Introduction

1.1 About the PrimeCell DMC (PL340) product range	1-2
1.2 Supported devices	1-6

Chapter 2

Functional Overview

2.1 DMC functional description	2-2
2.2 DMC functional operation	2-9

Chapter 3

Programmer's Model

3.1 About the programmer's model	3-2
3.2 Register summary	3-3
3.3 Register descriptions	3-6

Chapter 4

Programmer's Model for Test

4.1 Integration test registers	4-2
--------------------------------------	-----

Chapter 5	Device Driver	
	5.1 Device driver functions	5-2
Appendix A	Signal Descriptions	
	A.1 Clock and reset signals	A-2
	A.2 Miscellaneous signals	A-3
	A.3 AXI signals	A-4
	A.4 APB signals	A-9
	A.5 Pad interface signals	A-10
	Glossary	

List of Tables

PrimeCell Dynamic Memory Controller (PL340)

Technical Reference Manual

	Change history	ii
Table 1-1	Memory widths	1-3
Table 1-2	Memory widths and AXI port widths	1-4
Table 1-3	Attribute formats	1-5
Table 2-1	Example MDDR setup	2-25
Table 2-2	Valid system states for FSMs	2-28
Table 2-3	Recommended power states	2-29
Table 3-1	DMC register summary	3-5
Table 3-2	memc_status Register bit assignments	3-6
Table 3-3	memc_cmd Register bit assignments	3-8
Table 3-4	direct_cmd Register bit assignments	3-9
Table 3-5	memory_cfg Register bit assignments	3-10
Table 3-6	refresh_prd Register bit assignments	3-12
Table 3-7	cas_latency Register bit assignments	3-12
Table 3-8	t_dqss Register bit assignments	3-13
Table 3-9	t_mrd Register bit assignments	3-13
Table 3-10	t_ras Register bit assignments	3-14
Table 3-11	t_rc Register bit assignments	3-14
Table 3-12	t_rcd Register bit assignments	3-15
Table 3-13	t_rfc Register bit assignments	3-16
Table 3-14	t_rp Register bit assignments	3-16

Table 3-15	t_rrd Register bit assignments	3-17
Table 3-16	t_wr Register bit assignments	3-17
Table 3-17	t_wtr Register bit assignments	3-18
Table 3-18	t_xp Register bit assignments	3-18
Table 3-19	t_xsr Register bit assignments	3-19
Table 3-20	t_esr Register bit assignments	3-19
Table 3-21	id_<n>_cfg Registers bit assignments	3-20
Table 3-22	chip_<n>_cfg Registers bit assignments	3-21
Table 3-23	user_status Registers bit assignments	3-21
Table 3-24	user_status Registers bit assignments	3-21
Table 3-25	periph_id Register bit assignments	3-22
Table 3-26	periph_id_0 Register bit assignments	3-23
Table 3-27	periph_id_1 Register bit assignments	3-23
Table 3-28	periph_id_2 Register bit assignments	3-23
Table 3-29	periph_id_3 Register bit assignments	3-24
Table 3-30	comp_id Register bit assignments	3-25
Table 4-1	DMC test register summary	4-2
Table 4-2	int_cfg Register bit assignments	4-3
Table 4-3	int_inputs Register bit assignments	4-3
Table 4-4	int_outputs Register bit assignments	4-4
Table A-1	Clock and reset signals	A-2
Table A-2	Miscellaneous signals	A-3
Table A-3	Write address channel signals	A-4
Table A-4	Write data channel signals	A-5
Table A-5	Buffered write response channel signals	A-6
Table A-6	Read address channel signals	A-6
Table A-7	Read data channel signals	A-7
Table A-8	AXI low-power interface signals	A-8
Table A-9	APB signals	A-9
Table A-10	Pad interface signals	A-10

List of Figures

PrimeCell Dynamic Memory Controller (PL340)

Technical Reference Manual

	Key to timing diagram conventions	xii
Figure 1-1	Example DMC system	1-2
Figure 2-1	DMC block diagram	2-2
Figure 2-2	AXI slave interface connections	2-4
Figure 2-3	clk domain state diagram	2-5
Figure 2-4	AXI low-power interface channel signals	2-5
Figure 2-5	APB external connections	2-6
Figure 2-6	mclk domain FSM	2-7
Figure 2-7	Pad interface external connections	2-7
Figure 2-8	Command control output timing	2-20
Figure 2-9	Activate to read or write command timing, tRCD	2-20
Figure 2-10	Bank activate to bank activate or auto-refresh command timing, tRC	2-20
Figure 2-11	Bank activate to different bank activate for a memory timing, tRRD	2-21
Figure 2-12	Precharge to command and auto-refresh timing, tRP and tRFC	2-21
Figure 2-13	Activate to precharge, and precharge to precharge timing, tRAS and tRP	2-21
Figure 2-14	Mode register write to command timing, tMRD	2-22
Figure 2-15	Self-refresh entry and exit timing, tESR and tXSR	2-22
Figure 2-16	Power down entry and exit timing, tXP	2-22
Figure 2-17	Data output timing, tWTR	2-23
Figure 2-18	Data output timing, tDQSS = 1	2-23
Figure 2-19	Data input timing	2-24

Figure 2-20	System state transitions	2-30
Figure 3-1	Register map	3-2
Figure 3-2	DMC configuration register map	3-3
Figure 3-3	AXI ID configuration register map	3-4
Figure 3-4	Chip configuration registers map	3-4
Figure 3-5	Component configuration register map	3-4
Figure 3-6	memc_status Register bit assignments	3-6
Figure 3-7	memc_cmd Register bit assignments	3-7
Figure 3-8	direct_cmd Register bit assignments	3-8
Figure 3-9	memory_cfg Register bit assignments	3-9
Figure 3-10	refresh_prd Register bit assignments	3-11
Figure 3-11	cas_latency Register bit assignments	3-12
Figure 3-12	t_dqss Register bit assignments	3-13
Figure 3-13	t_mrd Register bit assignments	3-13
Figure 3-14	t_ras Register bit assignments	3-14
Figure 3-15	t_rc Register bit assignments	3-14
Figure 3-16	t_rcd Register bit assignments	3-15
Figure 3-17	t_rfc Register bit assignments	3-15
Figure 3-18	t_rp Register bit assignments	3-16
Figure 3-19	t_rrd Register bit assignments	3-17
Figure 3-20	t_wr Register bit assignments	3-17
Figure 3-21	t_wtr Register bit assignments	3-18
Figure 3-22	t_xp Register bit assignments	3-18
Figure 3-23	t_xsr Register bit assignments	3-19
Figure 3-24	t_esr Register bit assignments	3-19
Figure 3-25	id_<n>_cfg Registers bit assignments	3-20
Figure 3-26	chip_<n>_cfg Registers bit assignments	3-20
Figure 3-27	periph_id Register bit assignments	3-22
Figure 3-28	comp_id Register bit assignments	3-24
Figure 4-1	Integration Test Register map	4-2
Figure 4-2	int_cfg Register bit assignments	4-2
Figure 4-3	int_inputs Register bit assignments	4-3
Figure 4-4	int_outputs Register bit assignments	4-4

Preface

This preface introduces the *PrimeCell Dynamic Memory Controller (PL340) Revision r0p0 Technical Reference Manual*. It contains the following sections:

- *About this manual* on page x
- *Feedback* on page xiv.

About this manual

This is the Technical Reference Manual for the PrimeCell *Dynamic Memory Controller* (DMC) product range. The PrimeCell DMC product range comprises various preconfigured systems that support a single type and size of SDRAM on the memory interface.

Product revision status

The *rn**pn* identifier indicates the revision status of the product described in this manual, where:

- | | |
|-----------|--|
| rn | Identifies the major revision of the product. |
| pn | Identifies the minor revision or modification status of the product. |

Intended audience

This manual is written for implementation engineers and architects, and provides a description of an optimal DMC architecture. The DMC product range provides an interface between the *Advanced eXtensible Interface* (AXI) system bus and external (off-chip) memory devices.

Using this manual

This manual is organized into the following chapters:

Chapter 1 *Introduction*

Read this chapter for an introduction to the DMC product range and its features.

Chapter 2 *Functional Overview*

Read this chapter for an overview of the major functional blocks and the operation of the DMC product range.

Chapter 3 *Programmer's Model*

Read this chapter for a description of the registers.

Chapter 4 *Programmer's Model for Test*

Read this chapter for a description of the additional logic for integration testing.

Chapter 5 *Device Driver*

Read this chapter for a description of the device driver functions.

Appendix A *Signal Descriptions*

Read this appendix for a description of the DMC product range signals.

Glossary

Read the Glossary for definitions of terms used in this manual.

Conventions

Conventions that this manual can use are described in:

- *Typographical*
- *Timing diagrams* on page xii
- *Signals* on page xii
- *Numbering* on page xiii.

Typographical

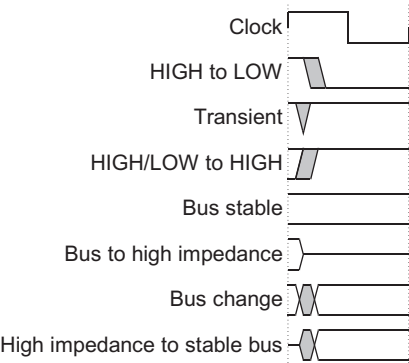
The typographical conventions are:

<i>italic</i>	Highlights important notes, introduces special terminology, denotes internal cross-references, and citations.
bold	Highlights interface elements, such as menu names. Denotes ARM processor signal names. Also used for terms in descriptive lists, where appropriate.
monospace	Denotes text that you can enter at the keyboard, such as commands, file and program names, and source code.
<u>monospace</u>	Denotes a permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<i>monospace italic</i>	Denotes arguments to monospace text where the argument is to be replaced by a specific value.
monospace bold	Denotes language keywords when used outside example code.
< and >	Angle brackets enclose replaceable terms for assembler syntax where they appear in code or code fragments. They appear in normal font in running text. For example: • MRC p15, 0 <Rd>, <CRn>, <CRm>, <Opcode_2> • The Opcode_2 value selects which register is accessed.

Timing diagrams

The figure named *Key to timing diagram conventions* explains the components used in timing diagrams. Variations, when they occur, have clear labels. You must not assume any timing information that is not explicit in the diagrams.

Shaded bus and signal areas are undefined, so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.



Key to timing diagram conventions

Signals

The signal conventions are:

Signal level	The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means HIGH for active-HIGH signals and LOW for active-LOW signals.
Lower-case n	Denotes an active-LOW signal.
Prefix A	Denotes global <i>Advanced eXtensible Interface</i> (AXI) signals:
Prefix AR	Denotes AXI read address channel signals.
Prefix AW	Denotes AXI write address channel signals.
Prefix B	Denotes AXI write response channel signals.
Prefix C	Denotes AXI low-power interface signals.
Prefix H	Denotes <i>Advanced High-performance Bus</i> (AHB) signals.

Prefix P Denotes *Advanced Peripheral Bus* (APB) signals.

Prefix R Denotes AXI read data channel signals.

Prefix W Denotes AXI write data channel signals.

Numbering

The numbering convention is:

<size in bits>'<base><number>

This is a Verilog method of abbreviating constant numbers. For example:

- 'h7B4 is an unsized hexadecimal value.
- 'o7654 is an unsized octal value.
- 8'd9 is an eight-bit wide decimal value of 9.
- 8'h3F is an eight-bit wide hexadecimal value of 0x3F. This is equivalent to b0011111.
- 8'b1111 is an eight-bit wide binary value of b00001111.

Further reading

This section lists publications by ARM Limited.

ARM Limited periodically provides updates and corrections to its documentation. See <http://www.arm.com> for current errata sheets, addenda, and the ARM Limited Frequently Asked Questions list.

ARM publications

This manual contains information that is specific to the DMC product range. See the following documents for other relevant information:

- *PrimeCell Dynamic Memory Controller (PL340) r0p0 Integration Manual* (ARM DII 0105)
- *PrimeCell Dynamic Memory Controller (PL340) r0p0 Implementation Guide* (ARM DII 0106)
- *AMBA AXI Protocol Specification* (ARM IHI 0022)
- *AMBA 3 APB Protocol Specification* (ARM IHI 0024)
- *ARM PrimeCell External Bus Interface (PL220) Technical Reference Manual* (ARM DDI 0249).

Feedback

ARM Limited welcomes feedback on the DMC product range and its documentation.

Feedback on this DMC product range

If you have any comments or suggestions about this product, contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this manual

If you have any comments on this manual, send e-mail to errata@arm.com giving:

- the title
- the number
- the relevant page number(s) to which your comments apply
- a concise explanation of your comments.

ARM Limited also welcomes general suggestions for additions and improvements.

Chapter 1

Introduction

This chapter introduces the DMC product range and contains the following sections:

- *About the PrimeCell DMC (PL340) product range* on page 1-2
- *Supported devices* on page 1-6.

1.1 About the PrimeCell DMC (PL340) product range

This book refers to the DMC product range as the DMC.

The DMC is an *Advanced Microcontroller Bus Architecture* (AMBA) compliant *System-on-Chip* (SoC) peripheral that is developed, tested, and licensed by ARM.

The PL340 product range is a range of high performance, area-optimized SDRAM memory controllers compatible with the AMBA AXI protocol.

Each member of the PL340 product range is pre-configured and validated with a number of options:

- the SDRAM memory type (see Table 1-1 on page 1-3)
- the number of SDRAM memory devices
- the maximum SDRAM memory width
- the number of outstanding AXI addresses.

The DMC supports the PrimeCell (PL220) *External Bus Interface* (EBI). This ensures that you can still use a shared external bus.

For more information on AMBA refer to:

- *AMBA AXI Protocol Specification*
- *AMBA 3 APB Protocol Specification*.

Figure 1-1 shows an example DMC system.

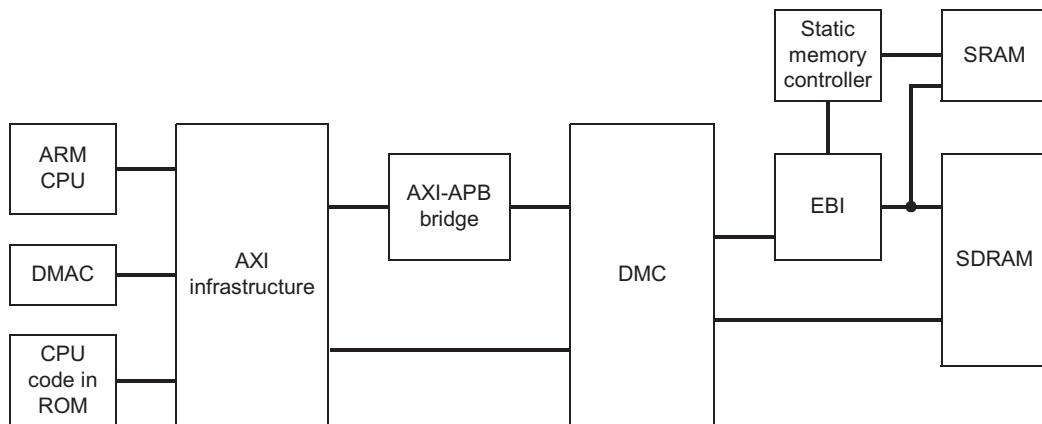


Figure 1-1 Example DMC system

1.1.1 Features of the DMC

The DMC block offers the following features:

- compatible with AMBA AXI for accesses to SDRAM
- compatible with AMBA APB for programming the DMC
- soft macrocell available in Verilog
- functional verification using a *File Reader Master* (FRM) test environment
- synchronous n:1 between AXI and APB
- synchronous or asynchronous operation between AXI bus infrastructure and external memory bus
- active and precharge power down supported in the SDRAM
- quality of service features for low latency transfers
- optimized utilization of external memory bus
- tie-off pin to select external memory widths
- multiple outstanding addresses
- write data interleaving supported
- preconfigured hardware resource to optimize area versus performance
- support for preconfigured number of ARMv6 architecture outstanding exclusive access transfers.

1.1.2 Supported memory widths

Table 1-1 lists the supported memory bus widths for different memory types.

Table 1-1 Memory widths

Memory bus width	SDR	Mobile DDR/DDR
16-bit	Yes	Yes
32-bit	Yes	Yes
64-bit	Yes	Yes

Table 1-2 lists the supported memory widths for different AXI interface widths.

Table 1-2 Memory widths and AXI port widths

Effective memory width ^a	AXI port width		
	32-bit	64-bit	128-bit
16-bit	Yes	No	No
32-bit	Yes	Yes	No
64-bit	Yes	Yes	Yes
128-bit	No	Yes	Yes

a. Effective memory width is the data transferred in a single clock cycle, that is 32 bits for 32-bit SDR and 32bits for 16-bit DDR.

Note

All memory configurations support 1-4 external memory chip selects.

Check the *Release Note* for the specific configuration of your controller.

1.1.3 AXI interface attributes

The slave interface has the following attributes:

Write Acceptance Capability

The maximum number of active write transactions that a slave can accept.

Read Acceptance Capability

The maximum number of active read transactions that a slave can accept.

Combined Acceptance Capability

The maximum number of active transactions that a slave can accept. Note that this indicated where the slave has combined read and write transaction storage so this is mutually exclusive to the write/read acceptance capability.

Write Interleave Depth

The number of active write transactions for which the slave can receive data. This is counted from the earliest transaction.

Read Data Reordering Depth

The number of active read transactions for which a slave may transmit data. This is counted from the earliest transaction.

Table 1-3 lists the attribute formats.

Table 1-3 Attribute formats

Attribute	Value
Combined acceptance capability	Arbiter queue depth
Write interleave depth	Write acceptance capability
Read data reorder depth	Read acceptance capability

1.2 Supported devices

See the product *Release Note* for more information.

Chapter 2

Functional Overview

This chapter describes the DMC operation. It contains the following sections:

- *DMC functional description* on page 2-2
- *DMC functional operation* on page 2-9

2.1 DMC functional description

Figure 2-1 shows a block diagram of the DMC.

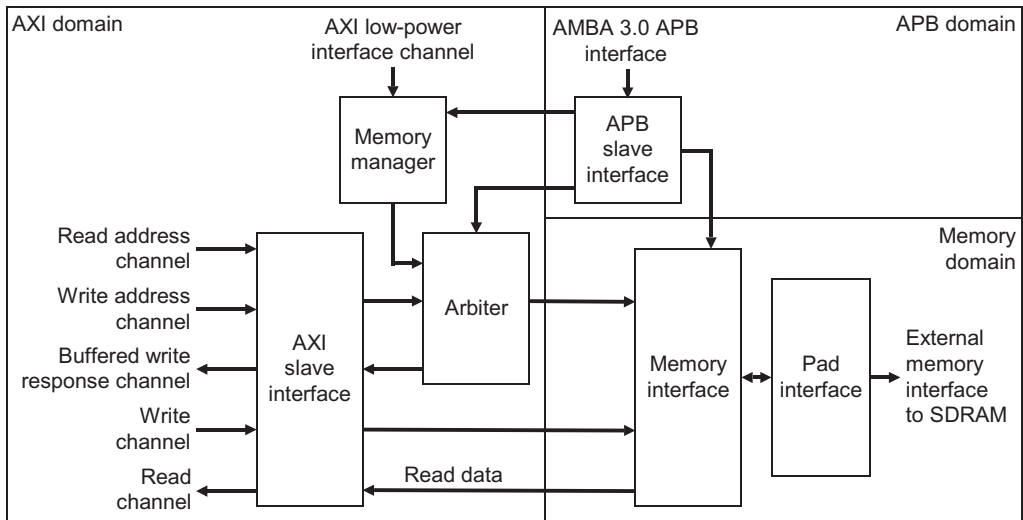


Figure 2-1 DMC block diagram

The main blocks in the DMC design are:

- *AXI slave interface*
- *Arbiter* on page 2-5
- *Memory manager* on page 2-5
- *APB slave interface* on page 2-6
- *Memory interface* on page 2-6
- *Pad interface* on page 2-7.

2.1.1 AXI slave interface

For detailed information about the AXI interface see the *AMBA AXI Protocol Specification*. The AXI slave interface comprises the following AXI channels:

AXI write address channel

Enable the transfer of the address and all other control data required for the DMC to carry out an AXI write transaction.

AXI write data channel Enables the transfer of write data and validating data byte strobes to the DMC.

AXI buffered write response channel

Enables the transfer of response information associated with an write transaction and a write data channel transaction.

AXI read address channel

Enable the transfer of the address and all other control data required for the DMC to carry out an AXI read transaction.

AXI read data channel

Enables the transfer of read data and response information associated with a read transaction.

Figure 2-2 on page 2-4 shows the AXI slave interface external connections.

Note

Reset signals are not shown on Figure 2-2 on page 2-4 to Figure 2-6 on page 2-7. See Table A-1 on page A-2 for details.

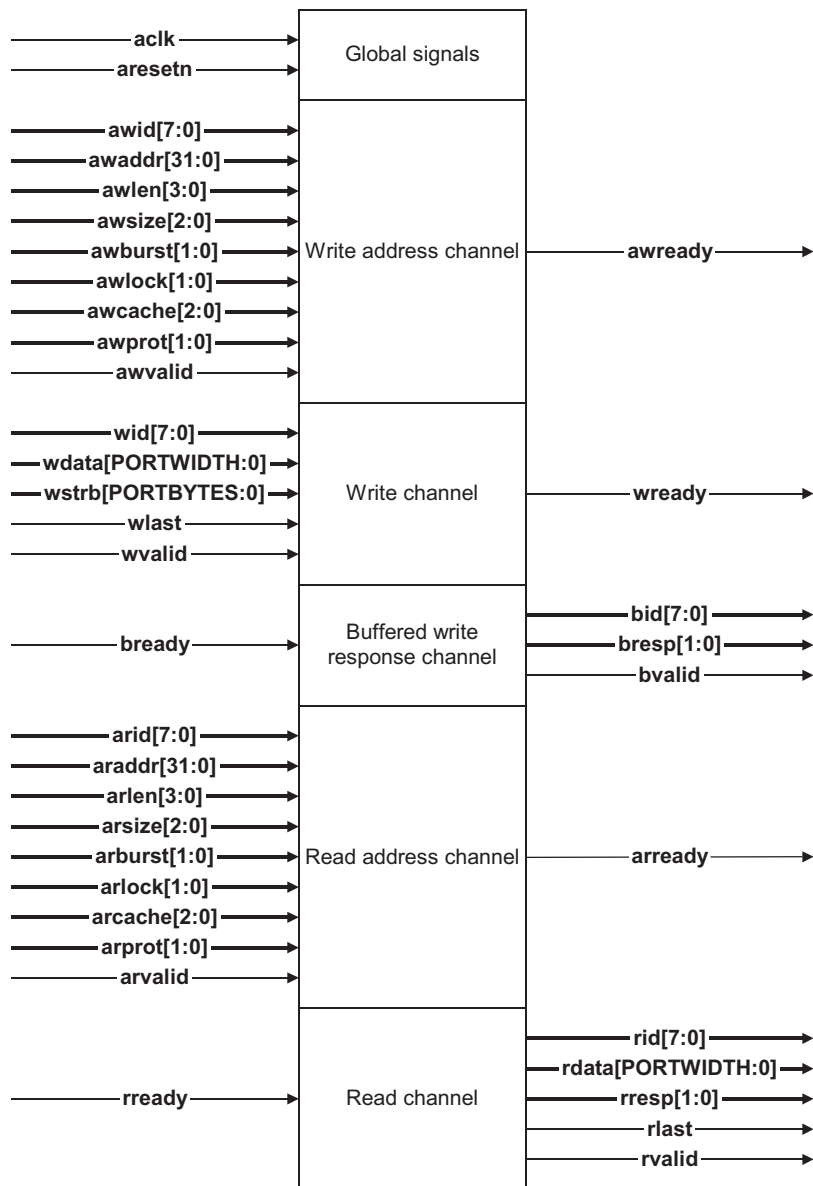


Figure 2-2 AXI slave interface connections

2.1.2 Arbiter

The arbiter receives memory access commands from the AXI slave interface and the memory manager. It passes the highest priority command to the memory interface after arbitration. Read data is passed from the memory interface to the AXI slave interface.

2.1.3 Memory manager

The memory manager tracks and controls the current state of the DMC using a *Finite State Machine* (FSM). See Figure 2-3.

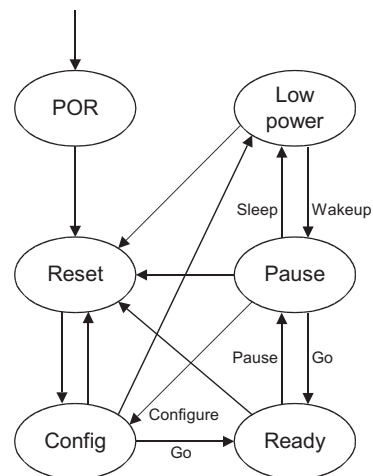


Figure 2-3 clk domain state diagram

In Figure 2-3 non-state moving transitions are omitted for clarity. See Table 2-2 on page 2-28 for valid system states.

APB commands to the Direct Command Register or the AXI low-power interface control the state of the DMC. Figure 2-4 shows AXI low-power interface channel signals.

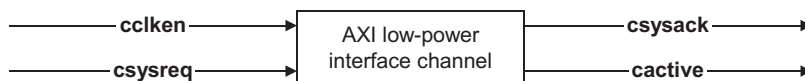


Figure 2-4 AXI low-power interface channel signals

The AXI low-power interface channel enables low-power mode to be entered using discrete lines. You can tie off this interface to be inactive if it is not required.

The APB slave interface stalls the **psel** and **penable** signals using the **pready** signal if a previous command has not completed.

If an APB command is received that is illegal to carry out from the current state then it is ignored and the FSM stays in the current state.

The memory manager enables the APB slave interface to directly send initialization commands to the external memory SDRAM and periodically generate refresh commands for the external memory SDRAM.

2.1.4 APB slave interface

The APB slave is a fully compliant APB slave. The DMC has 4KB of memory allocated to it.

For detailed information about the APB interface see the *AMBA 3 APB Protocol Specification*.

The APB interface implements the register file as described in Chapter 3 *Programmer's Model*.

———— Note ————

The APB only supports single-word 32-bit accesses. Bits [1:0] of the **paddr** signal are not used within the DMC, resulting in byte and half-word accesses being treated as word accesses.

Figure 2-5 shows the APB external connections.

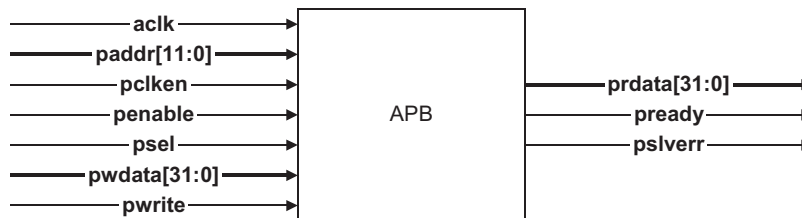


Figure 2-5 APB external connections

2.1.5 Memory interface

The memory interface provides a clean and defined interface between the pad interface and the arbiter ensuring the external memory interface command protocols are met in accordance with the programmed timings in the register block. See Chapter 3 *Programmer's Model*.

The only external I/Os to this block are **mclk** and **mresetn**.

The memory interface tracks and controls the state of the external memories using an FSM. See Figure 2-6.

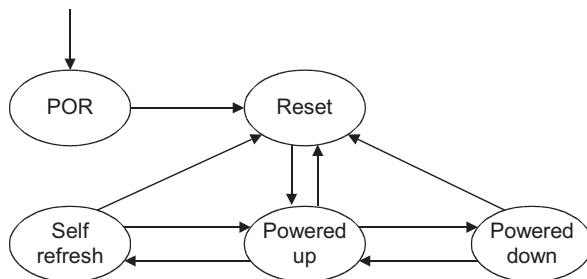


Figure 2-6 mclk domain FSM

See Table 2-2 on page 2-28 for valid system states.

2.1.6 Pad interface

The pad interface is a replaceable block depending on the type of memory you are connecting. It provides a flip flop for each external signal.

Figure 2-7 shows the pad interface external connections.

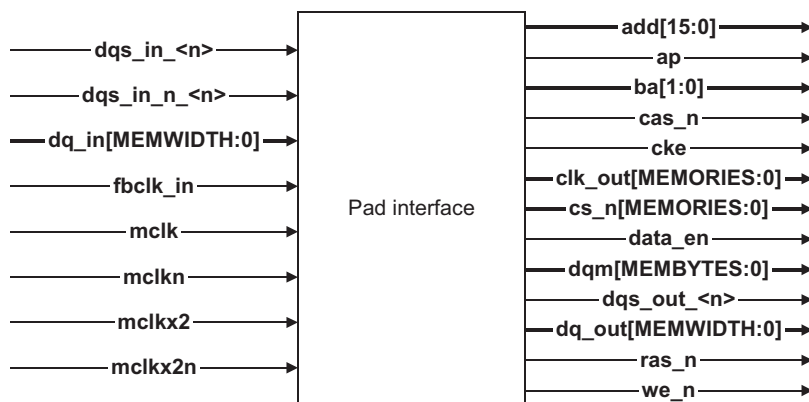


Figure 2-7 Pad interface external connections

Pad interface to external memory devices

It is possible that not all of the signals to the external memory devices are required. This depends on the memory type that the pad interface block supports.

The pad interface block registers the relevant command signals with clocks that enable the external memory device timing to be met.

To support a PrimeCell EBI (PL220) the **ap** (precharge bit) signal is also an external signal to the DMC. Having **ap** separate to the address bus means that PRECHARGEALL commands to dynamic memory can be carried out when the EBI has granted the external memory interface to another memory controller.

It is expected, for DDR memory devices, that a *Delay-Locked Loop* (DLL) is required to delay the **dqs** signals coming back from the memories with respect to the dq data bus. The standard delay for the DQS signals is a quarter clock period of **mcclk**. The DLL is not included in the DMC.

2.2 DMC functional operation

This section describes:

- *Clocking and resets*
- *Miscellaneous signals* on page 2-11
- *AXI slave interface* on page 2-12
- *Arbiter* on page 2-12
- *Memory manager* on page 2-17
- *APB slave interface* on page 2-18
- *Memory interface* on page 2-19
- *Pad interface* on page 2-24
- *Initialization* on page 2-24
- *Power-down support and usage model* on page 2-27.

2.2.1 Clocking and resets

This section describes:

- *Clocking*
- *Reset* on page 2-10.

Clocking

The DMC has the following functional clock inputs:

- **aclk**
- **mclk**
- **mclkn**
- **mclkx2**
- **mclkx2n**
- **fbclk**
- **dqs_in_<n>** where n is 0<n<bytes of external memory data bus
- **dqs_in_n_<n>** where n is 0<n<bytes of external memory data bus.

These clocks can be grouped into two clock domains:

aclk domain	aclk is in this domain. The aclk domain signals can only be stopped if the external memories are put in self refresh mode.
mclk domain	All clocks except aclk are in this domain. The mclk signal must be clocked at the rate of the external memory clock speed. The mclk domain signals can only be stopped if the external memories are put in self refresh mode.

See the *PrimeCell Dynamic Memory Controller (PL340) Integration Manual* for the required relationships of the clocks.

You can tie off the DMC **async** and **msync** pins so that the two clock domains can operate synchronously or asynchronously with respect to each other:

Synchronous clocking

The benefit of synchronous clocking is that you can reduce the read and write latency by removing the synchronization registers between clock domains. However, because of the integer relationship of the clocks, you might not be able to get the maximum performance from the system because of constraints placed on the bus frequency by the external memory clock speed. In synchronous mode the handshaking between the **ackl** and **mlkl** domains enables synchronous operation of the two clocks at multiples of each other, that is ratios of n:1 and 1:m.

Asynchronous clocking

The main benefit of asynchronous clocking is that you can maximize the system performance, while running the memory interface at a fixed system frequency. Additionally, in sleep-mode situations when the system is not required to do much work, you can lower the frequency to reduce power consumption.

Output clocks

A clock output is provided for every external memory device.

Reset

The DMC has two reset inputs:

aresetn This is the reset signal for the **ackl** domain.

mresetn This is the reset signal for the **mlkl** domain.

Both reset signals can be changed asynchronously to their respective clock domain. Internally to the DMC the deassertion of the **aresetn** signal is synchronized to **ackl**, and the deassertion of the **mresetn** signal is synchronized to the **mlkl**, **mlkln**, **mlkx2**, and **mlkx2n** clock signals. The clock inputs **fbclk**, **dqs_in_<n>**, and **dqs_in_n_<n>** do not clock any resettable logic.

2.2.2 Miscellaneous signals

You can use the following signals as general-purpose control signals for logic external to the DMC:

- user_status[7:0]** General-purpose ports that are readable from the APB interface. If you do not require these ports you must tie them either HIGH or LOW. These ports are connected directly to the APB interface block. Therefore, if they are driven from external logic that is not clocked by the DMC **ack** signal, then external synchronization registers are required.
- user_config[7:0]** General-purpose ports that are driven directly from a write-only APB register. If you do not require these ports leave them unconnected.

You can use the following miscellaneous signals as tie-offs to change the operational behavior of the DMC:

- cke_init** The **cke** output port to the external memory resets to this value.
- dqm_init** The **dqm** output ports to the external memory reset to this value.
- memory_width**

This configures the external memory width. See Table 3-2 on page 3-6 for the definition of these two pins. See the Release Note for legal values of memory width for each configuration.
- msync** When HIGH, indicates **mclk** is synchronous to **ack**. Otherwise they are asynchronous. Ensure that **msync** is tied to the same value as **async**.
- async** When HIGH, indicates **ack** is synchronous to **mclk**. Otherwise they are asynchronous. Ensure that **async** is tied to the same value as **msync**.
- use_ebi** When HIGH, indicates that the DMC must operate with a PrimeCell EBI. See *ARM PrimeCell External Bus Interface (PL220) Technical Reference Manual*.
- rst_bypass** Use this signal for ATPG testing only. It must be tied LOW for normal operation.
- dft_en_clk_out**

Use this signal for ATPG testing only. It must be tied LOW for normal operation.

2.2.3 AXI slave interface

The AXI programmer's view is of a flat area of memory. The full range of AXI operations are supported.

The base addresses of the external memory devices are programmable using the chip_cfg Registers, see *chip_<n>_cfg Registers* on page 3-20.

In addition to reads and writes, exclusive reads and writes are supported in accordance with the *AMBA AXI Protocol Specification*.

Successful exclusive accesses have an EXOKAY response. All other accesses, including exclusive fail accesses, receive an OKAY response.

Note

The **arcache** and **arprot** signals are included in the DMC port list for completeness only. There is no requirement for the DMC to use these signals.

Configuration options

You can choose 128, 64, or 32-bit data width. The default data width is 64-bit.

2.2.4 Arbiter

This section describes:

- *Formatting from AXI address channels* on page 2-13
- *Formatting from memory manager* on page 2-14
- *Arbiter access mux* on page 2-14
- *QOS* on page 2-14
- *Hazard detection* on page 2-15
- *Scheduler* on page 2-16
- *Arbitration algorithm* on page 2-16
- *Command formatting* on page 2-16
- *Exclusive access monitor* on page 2-16.

Formatting from AXI address channels

Formatting is as follows:

Chip select decoding

Using the programmed values in the chip_<n>_cfg Registers defined in Chapter 3 *Programmer's Model*, an incoming address has the most significant eight address bits compared with the address match bits using the address mask to ignore any don't care bits to select an external chip.

The transfer is still carried out if there is no match but the result is undefined.

Row select decoding

The row address is determined from the AXI address using bits [5:3] of the memory_cfg Register, and also the brc_n_rbc bit for the selected chip defined in the chip_<n>_cfg Register.

Column select decoding

The column address is determined from the AXI address using bits [2:0] of the memory_cfg Register.

Bank select decoding

The chip bank is determined using the brc_n_rbc bit for the selected chip defined in the chip_<n>_cfg Register.

Number of beats

The number of memory beats is determined, depending on the effective external memory width and the burst size of the AXI access. AXI wrapping bursts are split into two incrementing bursts.

Quality Of Service (QOS) selection

QOS is defined for the DMC as a method of increasing the arbitration priority of a read access that requires low-latency read data. The quality of service for an AXI read access is determined when it is received by the arbiter. There is no quality of service for write accesses.

There are two forms of quality of service tracking:

- qos_max time-out
- qos_min time-out.

The following example shows that the type of QOS and QOS value is determined by the id_<n>_cfg Register values pointed to from the **ARID** bits selected by the qos_master_bits in the memory_cfg Register. See *Memory Configuration Register* on page 3-9.

If a transfer is received that has an **ARID** of 0x5A and the qos_master_bits in the memory_cfg Register are set to b010 then the four ID bits used to match to a QOS pointer are **ARID[5:2]** giving a value 0x6.

This means that when the transfer is received the id_6_cfg Register determines the qos_enable, qos_min, and qos_max values for this transfer. If the qos_enable bit is **HIGH** then the new arbiter entry that is created for this transfer is assigned the qos_min value and qos_max value from the id_6_cfg Register. In addition to this, if the qos_override bit associated with this transfers ID, qos_override[6], is **HIGH** when the **arvalid** and **arready** signals are **HIGH** then the qos_min arbiter entry is forced **HIGH** irrespective of whether the qos_enable bit is **HIGH**.

Formatting from memory manager

The direct command uses the chip select bits [21:20] to access the memory chip.

The command to be carried out is either an auto refresh, or from the APB interface. It is encoded by the memory manager to match the format required by the arbiter.

Arbiter access mux

The selection of a command from the AXI interface or the memory manager is fixed, with the memory manager having a higher priority.

The selection between an AXI read access and AXI write access is made using a round-robin arbitration, unless a read access has a low latency QOS value, see *Formatting from AXI address channels* on page 2-13. In this case it is arbitrated immediately.

QOS

For write accesses, no quality of service is provided.

See *Formatting from AXI address channels* on page 2-13 for how the quality of service is selected for an AXI read access.

If the QOS enable bit for the **ARID** is set in the register bank, the QOS maximum latency value is decremented every cycle until it reaches zero.

If the entry is still in the queue when the QOS maximum latency value reaches zero then the entry becomes high priority. This is called a *time-out*. Also, any entry in the queue with a minimum latency QOS also produces a time-out. Minimum latency time-outs have priority over maximum latency time-outs.

When an entry times out in this way it forces a time-out onto any entries that it has dependencies against. In normal operation, these entries will have already timed out because they would have received the same initial QOS value, but been decrementing for longer. The highest priority arbiter entry is serviced next.

One special case exists. This is when or if the `qos_override` function forces a minimum latency time-out. In this instance, any accesses that the new entry has dependencies against might not have timed out and are forced to time out so that the high-priority entry can start as soon as possible.

A QOS is also provided for the auto-refresh commands from the memory manager. The arbiter keeps track of the number of auto-refresh commands in the arbiter queue with a simple increment-decrement counter. If the number of auto-refresh commands reaches a set limit of six, a refresh time-out is signalled to the arbiter queue. This forces all of the auto-refresh queue entries to have a time-out. This time-out is sticky, and does not disappear when the number of time-outs drops back below the threshold. Instead it remains asserted until all of the auto-refreshes have been serviced. This provides a guaranteed refresh rate in the SDRAM.

Hazard detection

There are two types of hazard:

Read After Read (RAR)

There is a read already in the arbiter queue with the same ID as the incoming entry, which is also a read.

Write After Write (WAW)

There is a write already in the arbiter queue with the same ID as the incoming entry, which is also a write.

The arbiter entry is flagged as having a dependency if a hazard is detected. There might be dependencies against a number of other arbiter entries. As the arbiter entries are invalidated, so the dependencies are reduced until finally there are no outstanding dependencies and the entry is free to start.

Note

There are no *Read-After-Write* (RAW) or *Write-After-Read* (WAR) hazard checks in the DMC. If an AXI master requires ordering between reads and writes to certain memory locations, it must wait for a buffered write response before issuing a read from a location it has written to (RAW). Also, it must wait for read data before issuing a write to a location it has read from (WAR).

Scheduler

The scheduler keeps track of the activity of the bank FSMs in the memory interface. This enables the arbiter to select an entry from the queue that does not stall the memory pipeline.

Arbitration algorithm

This is combinatorial logic to select the read pointer from the current queue state.

The ordering of commands to be carried out from the arbiter queue is arbitrated with a priority scheme of the following order:

- read min-latency timeout
- read max-latency timeout.

If the last command was an open-row access:

- next access to a bank, for which there are no pending open-row accesses, that is an open-row miss.

If the last open-row access was a read:

- open-row read
- open-row write.

Else:

- open-row write
- open-row read
- open-row miss.

Command formatting

For every memory burst access necessary to complete an arbiter queue entry, a memory interface command is required.

Command formatting is used to calculate the number of memory interface commands and memory cycles for each command to complete the next arbiter queue entry that is to be sent to the memory interface. It contains an address incrementor and a beat decrementor so that the arbiter entry can be interrupted and restarted.

Exclusive access monitor

This keeps track of a configurable number of outstanding exclusive accesses, up to four as described by the *AMBA AXI Protocol Specification*. If an exclusive write fails, the data mask for the write is forced LOW, so the data is not written.

When monitoring an exclusive access, the address of any write from another master is compared with the monitored address to check that the location is not being updated.

If the write crosses an address boundary then all of the least-significant address bits up to that boundary are not compared. Example 2-1 provides information about three transactions.

Example 2-1

Exclusive Read	Address = 0x000, size = WORD, length = 1, ID = 0.
Write	Address = 0x004, size = WORD, length = 2, ID = 1.
Exclusive Write	Address = 0x000, size = WORD, length = 1, ID = 0.

The DMC marks the exclusive write as having failed because the write crosses the 0x010 address boundary. The write, for comparison purposes, has therefore accessed the region 0x000-0x01f because bits 4:0 have not been compared.

The burst type is always taken to be INCR when calculating the address region accessed by the write. Therefore, a wrapped transaction in Example 2-1 that wraps down to 0x0 rather than cross the boundary, is treated in the same way. This is the same for a fixed burst that does not cross the boundary or wrap down to 0x0.

2.2.5 Memory manager

The memory manager issues commands from one of the following sources:

Direct commands

These are received over the APB interface as a result of a write to the `direct_cmd` Register (see *Direct Command Register* on page 3-8). They are used to initialize the SDRAM. The only valid commands that the memory manager can handle are:

- NOP
- PRECHARGEALL
- AUTOREFRESH
- MODEREG
- EXTENDED MODEREG.

Commands from the status FSM

The memory manager status FSM can be traversed by writing to the `memc_cmd` Register. The status FSM states can only be traversed when the DMC is idle. For example, the ready state can only be entered from the Config state when all direct commands have been completed.

Refresh commands

The refresh FSM can issue commands to the arbiter to refresh the SDRAM chips. The refresh counter is clocked by the memory clock to enable the frequency of the DMC to be scaled without affecting the refresh rate. The refresh rate period is programmable using the RefreshPrd Register. The value of this register is the count value in **mclk** cycles.

When the refresh counter wraps around zero an individual auto-refresh sequence is requested for each external chip in turn.

Programmable options

Auto Refresh Request Period (RefreshPrd) is the time period in **mclk** clock cycles that the memory manager generates a request for the arbiter to generate an auto-refresh command. This request is arbitrated as another command and is not necessarily initiated immediately. See *Arbiter* on page 2-12.

2.2.6 APB slave interface

The APB interface is clocked by the same clock as the AXI domain clock, **aclk**, but has a clock enable so that it can be slowed down to execute at an integer divisor of **aclk**.

To enable a clean registered interface to the external infrastructure the APB interface always adds a wait state for all reads and writes by driving **pready** LOW. In the following instances a delay of more than one wait state can be generated:

- when a direct command is received and there are outstanding commands that prevent a new command being stored in the command FIFO
- when a memory command is received and a previous memory command has not been completed.

The only registers that can be accessed when the DMC is not in the Config or Low-power state are the Memory Controller Status Register, to read the current state, and the Memory Controller Command Register to change state.

To guarantee no missed auto-refresh commands it is recommended that any change of **mclk** period and therefore update of the refresh period is carried out when the DMC is in the low-power state. This is because the refresh rate is dependent upon the **mclk** period. It is recommended that direct commands to the external memories are only written when the DMC is in the Config state and not in the Low-power state.

2.2.7 Memory interface

The interface is separated from the arbiter using three configurable synchronous or asynchronous FIFOs:

- command FIFO
- read data FIFO
- write data FIFO.

There is also a static interface that has configuration signals that cannot be changed when the interface is operating.

The memory interface reads commands from the arbiter using a FIFO but only when that command can be executed. The memory interface ensures a command is only executed when all the inter-command delays, defined in this section, for that bank or chip are met. The memory interface enables multiple banks to be active at any one time. However, only one bank can be carrying out a data transfer at any one time. If the command at the head of the FIFO cannot be executed then the command pipeline stalls until it can be executed.

The timing parameters in Figure 2-8 on page 2-20 to Figure 2-19 on page 2-24 can all be programmed using the APB interface. See Chapter 3 *Programmer's Model*.

When the `auto_power_down` Register bit is set then the **cke** output pin is negated to take the external memories into active or precharge power down depending on whether there is a row open. See Figure 2-16 on page 2-22. When exiting power down mode the delay before the next command is issued is defined by the register value `tXP`.

There is an FSM to control the operation of the power down mode. This FSM has a state that is entered when the SDRAM is put into self refresh mode. This is used so that if power is removed from all of the DMC apart from the memory interface and pad interface the state of the memory is known. When the rest of the DMC is powered up the status FSM enters the Low-power state rather than the Config state.

Memory interface to pad interface timing

All command control outputs are clocked on the same edge. In Figure 2-8 on page 2-20 to Figure 2-19 on page 2-24 the control outputs to the external memory are always clocked on the falling edge of the memory clock.

The relative times between control signals from the memory interface are maintained when output from the pad interface to the actual SDRAM devices. Therefore the timing register values required for a particular SDRAM device can be determined from that SDRAM device's data sheet. Figure 2-8 on page 2-20 to Figure 2-19 on page 2-24 show how the data sheet timings map onto the DMC timing registers.

The times in Figure 2-8 to Figure 2-19 on page 2-24 are not necessarily the default timing values but are values that are small enough to show the entire delay in one figure.

———— **Note** ————

The `command_en`, `data_cntl_en`, and `read_en` signals are internal to the DMC.

Figure 2-8 shows the command control output timing.

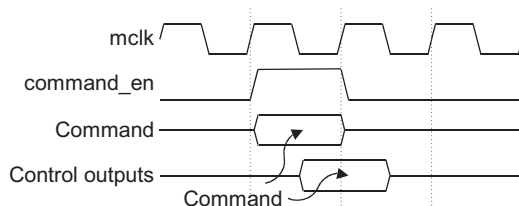


Figure 2-8 Command control output timing

Figure 2-9 shows the activate to read or write command timing.

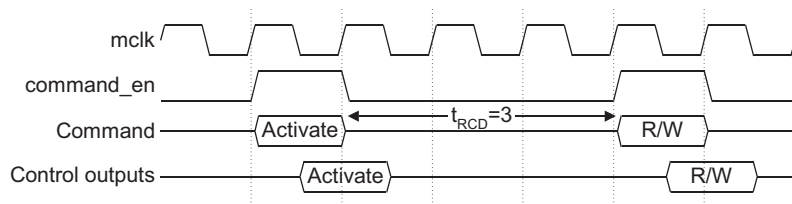


Figure 2-9 Activate to read or write command timing, t_{RCD}

Figure 2-10 shows the bank activate to bank activate or autorefresh command timing.

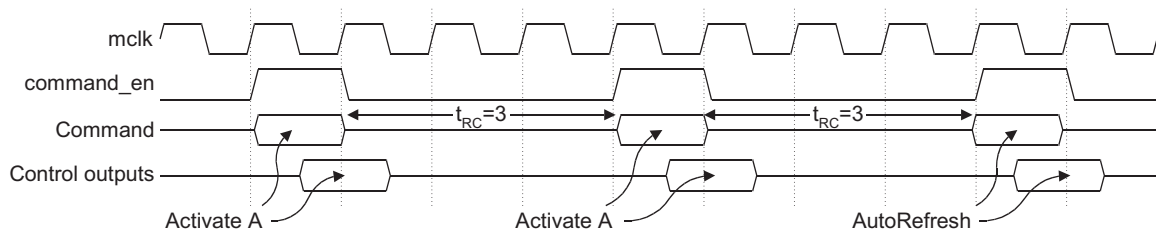


Figure 2-10 Bank activate to bank activate or auto-refresh command timing, t_{RC}

Figure 2-11 on page 2-21 shows the bank activate to different bank activate for a memory timing.

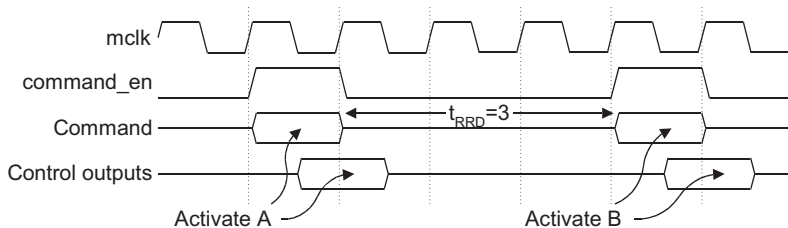


Figure 2-11 Bank activate to different bank activate for a memory timing, t_{RRD}

Figure 2-12 shows the precharge to command and auto-refresh timing.

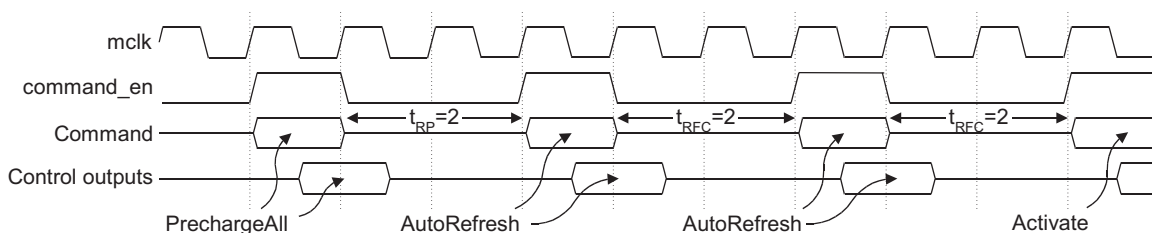


Figure 2-12 Precharge to command and auto-refresh timing, t_{RP} and t_{RFC}

Figure 2-13 shows activate to precharge, and precharge to precharge timing.

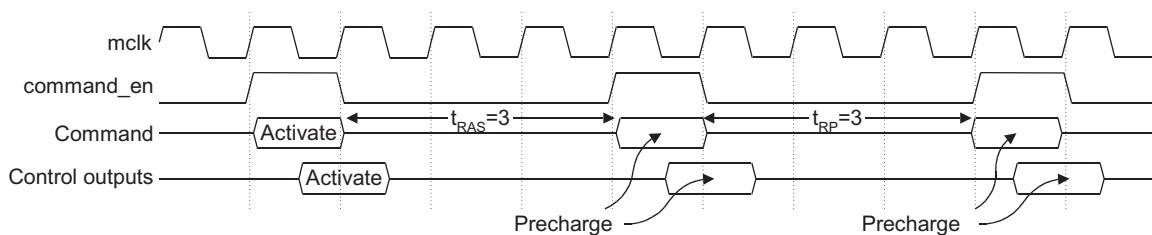


Figure 2-13 Activate to precharge, and precharge to precharge timing, t_{RAS} and t_{RP}

Figure 2-14 on page 2-22 shows mode register write to command timing.

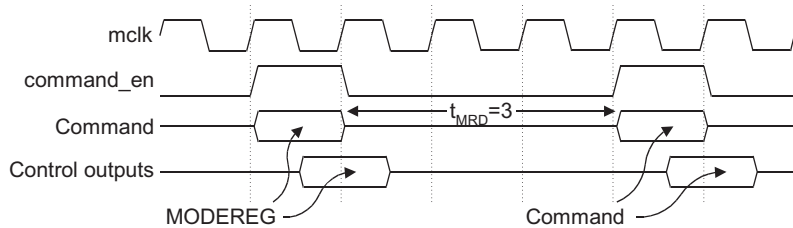


Figure 2-14 Mode register write to command timing, t_{MRD}

Figure 2-15 shows self-refresh entry and exit timing.

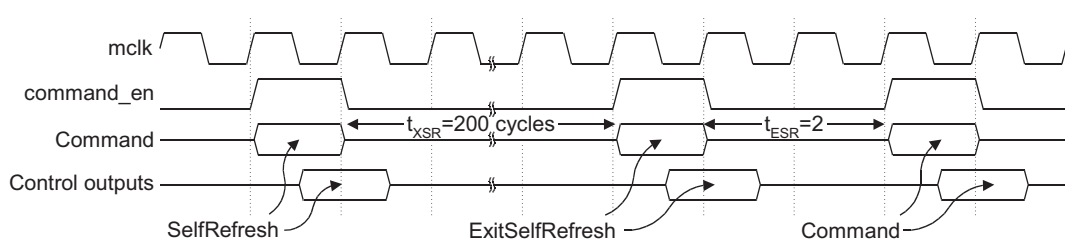


Figure 2-15 Self-refresh entry and exit timing, t_{ESR} and t_{XSR}

Figure 2-16 shows power down entry and exit timing.

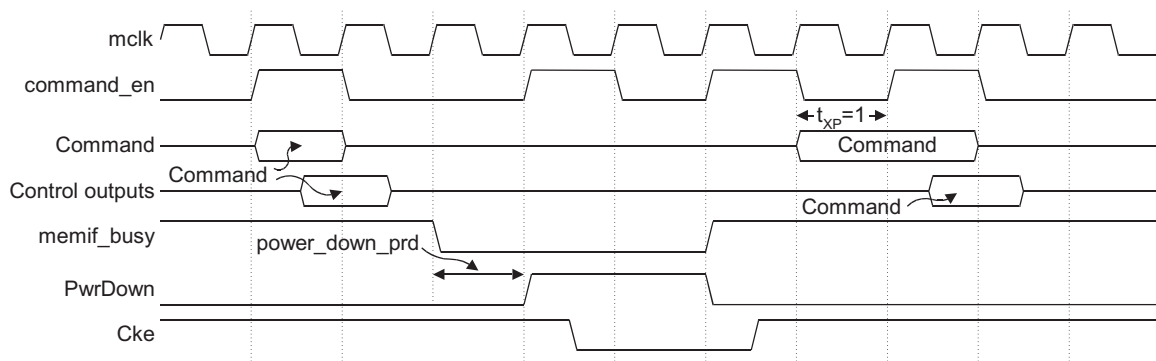


Figure 2-16 Power down entry and exit timing, t_{XP}

The **pwr_down_prd** count is timed from the memory interface becoming idle, that is after a command delay has timed out or the read data FIFO is emptied. **cke** is asserted when the command FIFO is not empty.

Figure 2-17 shows the turnaround time, t_{WTR} , for the memory interface to output a Write command followed immediately by a Read command.

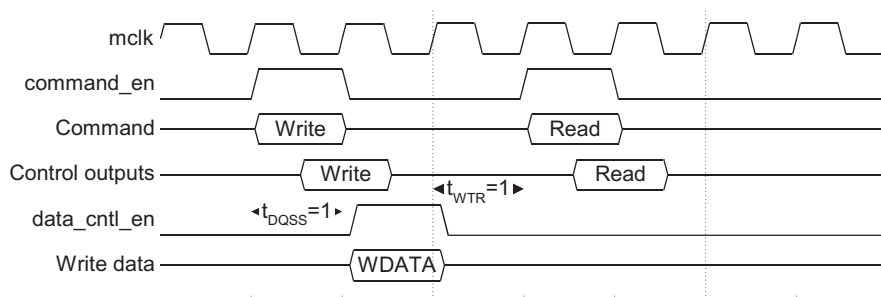


Figure 2-17 Data output timing, t_{WTR}

Figure 2-18 shows the relationship between memory interface outputting the Write command and the WDATA when t_{DQSS} is set to 1. It also highlights the t_{WR} minimum time between a Write and a Precharge command.

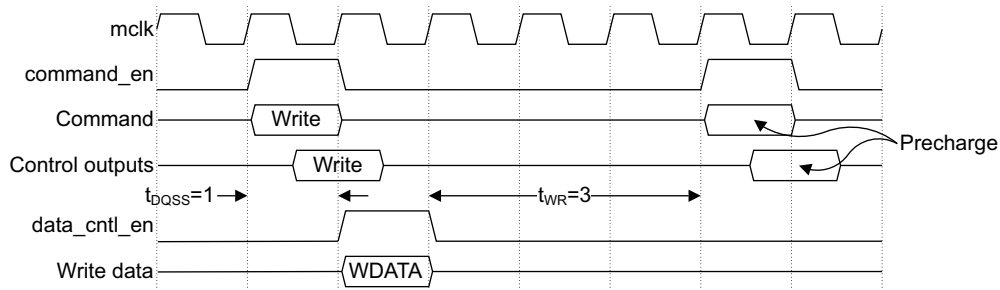


Figure 2-18 Data output timing, $t_{DQSS} = 1$

Figure 2-19 on page 2-24 shows the timing relationship between the Read command being output from the memory interface and the RDATA being returned to the memory interface from the pad interface.

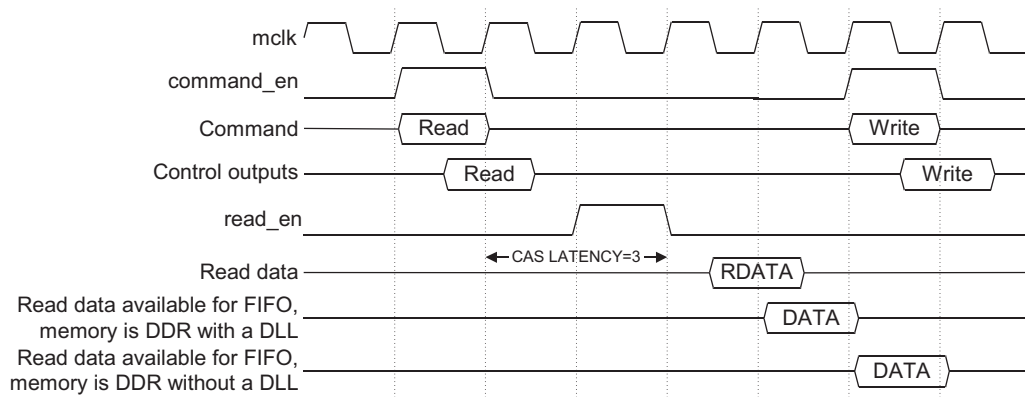


Figure 2-19 Data input timing

2.2.8 Pad interface

You can replace or modify the pad interface if required for further optimization for a particular memory type or target library or to use a hard macro.

It is important if the pad interface is modified or replaced that the relative timings of the control output signals enabled by **command_en** and **data_cntl_en** signals are maintained to ensure the timings carried out in the memory interface block are still correct at the external memory bus interface. The **read_en** signal is always asserted one **mclk** period before the expected read data. Therefore, the timing of **read_en** changes as CAS latency is changed using the APB interface.

When read commands are being executed the data is clock enabled into a FIFO clocked by **mclk** a set time from the command being clocked out. See Figure 2-19. This delay is dependent on the current **cas_latency** value programmed and the delays in the pad interface.

2.2.9 Initialization

Before you can use the DMC operationally to access external memory, you must:

- set up the DMC configuration registers
- initialize the external memory devices.

You might not have to configure all the DMC registers because some might power up to the correct value, see Chapter 3 *Programmer's Model*. For completeness all register values are included in Table 2-1 on page 2-25.

Note

You might create a deadlock situation if the DMC AXI port is accessed by a master before that master has configured the DMC using the APB port. A master that cannot access the APB port but accesses the AXI port before the DMC has been configured is held off until another master configures the DMC.

Example setup

Table 2-1 shows an example MDDR setup.

Table 2-1 Example MDDR setup

Register address	Write data	Description
0x80000014	0x00000006	Set cas_latency to 3
0x80000018	0x00000001	Set t_dqs to 1
0x8000001C	0x00000002	Set t_mrd to 2
0x80000020	0x00000007	Set t_ras to 7
0x80000024	0x0000000B	Set t_rc to 11
0x80000028	0x00000015	Set t_rcd to 5 and schedule_rcd to 2
0x8000002C	0x000001F2	Set t_rfc to 18 and schedule_rfc to 15
0x80000030	0x00000015	Set t_rp to 5 and schedule_rp to 2
0x80000034	0x00000002	Set t_rrd to 2
0x80000038	0x00000003	Set t_wr to 3
0x8000003C	0x00000002	Set t_wtr to 2
0x80000040	0x00000001	Set t_xp to 1
0x80000044	0x0000000A	Set t_xsr to 10
0x80000048	0x00000014	Set t_esr to 20
0x8000000C	0x000D0020	Set memory configuration ^a
0x80000010	0x00000A60	Set auto refresh period to be every 2656 mclk periods
0x80000200	0x000000FF	Set chip select for chip 0 to be 0x00XXXXXX, rbc configuration

Table 2-1 Example MDDR setup (continued)

Register address	Write data	Description
0x80000204	0x000022FF	Set chip select for chip 1 to be 0x22XXXXXX, rbc configuration
0x80000208	0x000055FF	Set chip select for chip 2 to be 0x55XXXXXX, rbc configuration
0x8000020C	0x00007FFF	Set chip select for chip 3 to be 0x7FXXXXXX, rbc configuration
0x80000008	0x000C0000	Carry out chip0 Nop command
0x80000008	0x00000000	Carry out chip0 Prechargeall command
0x80000008	0x00040000	Carry out chip0 Autorefresh command
0x80000008	0x00040000	Carry out chip0 Autorefresh command
0x80000008	0x00080032	Carry out chip0 Mode Reg command 0x32 mapped to low add bits
0x80000008	0x001C0000	Carry out chip1 Nop command
0x80000008	0x00100000	Carry out chip1 Prechargeall command
0x80000008	0x00140000	Carry out chip1 Autorefresh command
0x80000008	0x00140000	Carry out chip1 Autorefresh command
0x80000008	0x00180032	Carry out chip1 Mode Reg command 0x32 mapped to low add bits
0x80000008	0x002C0000	Carry out chip2 Nop command
0x80000008	0x00200000	Carry out chip2 Prechargeall command
0x80000008	0x00240000	Carry out chip2 Autorefresh command
0x80000008	0x00240000	Carry out chip2 Autorefresh command
0x80000008	0x00280032	Carry out chip2 Mode Reg command 0x32 mapped to low add bits
0x80000008	0x003C0000	Carry out chip3 Nop command
0x80000008	0x00300000	Carry out chip3 Prechargeall command
0x80000008	0x00340000	Carry out chip3 Autorefresh command
0x80000008	0x00340000	Carry out chip3 Autorefresh command
0x80000008	0x00380032	Carry out chip3 Mode Reg command 0x32 mapped to low add bits
0x80000004	0x00000000	Change DMC state to Ready

- a. The memory is configured as follows:
 - 8 column bits, 11 row bits
 - precharge all bit is shared with A10
 - power-down period of 0
 - auto power down is disabled
 - dynamic clock stopping is disabled
 - memory burst size of 4
 - **ARID[5:2]** bits to be used for quality of service.

2.2.10 Power-down support and usage model

The DMC provides architectural support for low-power operation in three ways:

- By using the `memory_cfg` Register at address offset `0xC`. The DMC can automatically place the SDRAM into either the precharge power-down or active power-down states by deasserting **cke** when the SDRAM has been inactive for a number of clock cycles. This occurs with the DMC in its **READY** state.
- By using the `memc_cmd` and `memc_status` Registers at address offsets `0x4` and `0x0`. The DMC can place the SDRAM into the self-refresh state under software control.
- By using the AXI low-power interface, the DMC can place the SDRAM into the self-refresh state under hardware control.

Additionally, the DMC microarchitecture provides further power savings through extensive use of clock gating. This includes clock gating of the external memory clocks by selecting the `stop_mem_clock` bit in the `mem_cfg` Register.

It is possible to implement DMC with two power domains:

- APB and AXI, **aclk**
- memory, **mclk**.

See Figure 2-1 on page 2-2.

Table 2-2 shows the valid system states of the **aclk** domain FSM and the **mclk** domain FSM. It also shows the valid power, clock and reset states in the **aclk** and **mclk** domains. Table 2-3 on page 2-30 shows the valid transitions, and the text following it explains how to traverse the system states.

Table 2-2 Valid system states for FSMs

SDRAM		DMC								System state
		aclk FSM				mclk FSM				
V _{DD}	State	V _{DD}	Clock	Reset	State	V _{DD}	Clock	Reset	State	
0	Null	0	N/a	N/a	Null	0	N/a	N/a	Null	1
0	Null	>0	Running	No	POR	>0	Running	No	POR	2
0	Null	>0	Running	Yes	Reset	>0	Running	Yes	Reset	3
0	Null	>0	Running	No	Config	>0	Running	No	Powered_up	4
>0	Accessible	>0	Running	No	Config	>0	Running	No	Powered_up	5
>0	Accessible	>0	Running	No	Ready	>0	Running	No	Powered_up	6
>0	Powered-down	>0	Running	No	Ready	>0	Running	No	Powered_down	7
>0	Self_refresh	>0	Running	No	Low_power	>0	Running	No	Self_refresh	8
>0	Self_refresh	>0	Running	No	Low_power	>0	Stopped	No	Self_refresh	9
>0	Self_refresh	>0	Stopped	No	Low_power	>0	Running	No	Self_refresh	10
>0	Self_refresh	>0	Stopped	No	Low_power	>0	Stopped	No	Self_refresh	11
>0	Self_refresh	0	N/a	N/a	Null	>0	Stopped	No	Self_refresh	12
>0	Self_refresh	0	N/a	N/a	Null	>0	Running	No	Self_refresh	13
>0	Self_refresh	>0	Running	No	POR	>0	Stopped	No	Self_refresh	14

Table 2-2 Valid system states for FSMs (continued)

SDRAM		DMC								System state
		aclk FSM				mclk FSM				
V _{DD}	State	V _{DD}	Clock	Reset	State	V _{DD}	Clock	Reset	State	
>0	Self_refresh	>0	Running	No	POR	>0	Running	No	Self_refresh	15
>0	Self_refresh	>0	Running	Yes	Reset	>0	Stopped	No	Self_refresh	16
>0	Self_refresh	>0	Running	Yes	Reset	>0	Running	No	Self_refresh	17

The ranking of system power states, from highest power to lowest power is as follows: 6, 7, 8, 10, 9, 11, 13, 12. However, states 8-11 are similar and the recommendation is to use state 11 from this group if clock-stopping techniques are available. Similarly, states 12 & 13 are similar and the recommendation is to use state 12 from this pair. Table 2-3 shows a recommended set of power states.

Table 2-3 Recommended power states

System state	Power name
6	Running
7	Auto power-down
11	Shallow self-refresh
12	Deep self refresh

These states and arcs are highlighted in Figure 2-20 on page 2-30.

Note

States 1-5, 9, 14, and 16 are only used as transitional states.

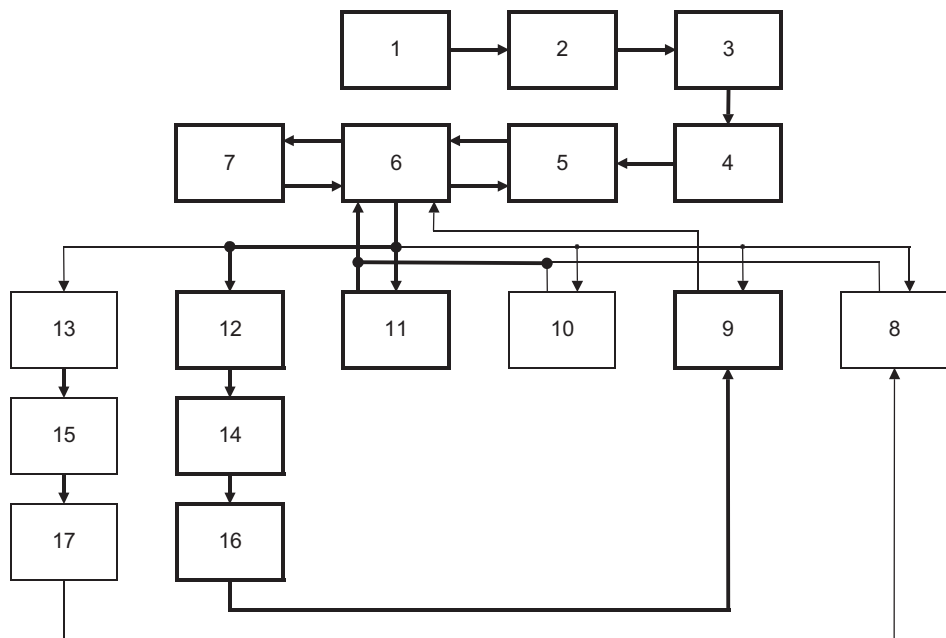


Figure 2-20 System state transitions

State transitions are as follows:

- Arc 1 to 2** Apply power to all DMC power domains, and ensure that **aclk** and **mclk** are running.
- Arc 2 to 3** Assert reset in both the **aclk** reset domain and the **mclk** reset domain.
- Arc 3 to 4** Deassert reset in both the **aclk** reset domain and the **mclk** reset domain.
- Arc 4 to 5** Apply power to the SDRAM power domain.
- Arc 5 to 6** You must:
1. Write to all of the memory timing parameters, address offsets 0x14 to 0x44.
 2. Write to the memory_cfg and refresh_prd Registers, address offsets 0xC and 0x10.
 3. Initialize the memory, using the direct_cmd Register, offset 0x8, with the sequence of commands specified by the memory vendor. When you have sent these commands to the memory, you can write to the memc_cmd Register, offset 0x4 with the GO command, 0x0.

4. Poll the memc_status Register until the value of 0x1 is returned, READY, signifying that the DMC is ready to accept AXI accesses to the SDRAM.

Arc 6 to 5 If you want to reconfigure either the DMC or SDRAM, you must first write to the memc_cmd Register, offset 0x4, with the Pause command, 0x3, and poll the memc_status Register until the value of 0x2 is returned, Paused. Then you can write to the memc_cmd Register with the Configure command, 0x4 and poll the memc_status Register until the value of 0x0 is returned, Config.

Arc 6 to 7 If auto_power_down is set in the memory_cfg Register (see *Memory Configuration Register* on page 3-9) then this arc is automatically taken when the SDRAM has been idle for power_down_prd **mclk** cycles.

Arc 7 to 6 When an SDRAM access command has been received in the **mclk** domain, this arc is taken.

Arc 6 to 8 You can take this arc under either hardware or software control:

- To take this arc under software control:
 1. Issue the Pause command.
 2. Poll for the Paused state.
 3. Issue the Sleep command.
- To take this arc under hardware control, use the AXI low-power interface to request a low-power state.

Arc 6 to 9 The same as arc 6 to 8, but additionally stop the **mclk** domain clock.

Arc 6 to 10 The same as arc 6 to 8, but additionally stop the **acclk** domain clock.

Arc 6 to 11 The same as arc 6 to 8, but additionally stop both the **mclk** and the **acclk** domain clocks.

Arc 6 to 12 The same as arc 6 to 8, but additionally stop the **mclk** domain clock and remove power from the **acclk** power domain. This can only be done if the DMC implementation has separate power domains for **acclk** and **mclk**.

Arc 6 to 13 The same as arc 6 to 8, but additionally remove power from the **acclk** power domain. This can only be done if the DMC implementation has separate power domains for **acclk** and **mclk**.

Arc 8 to 6 You can take this arc under either hardware or software control:

- To take this arc under software control:
 1. Issue the Wakeup command to the memc_cmd Register.

2. Poll the memc_status Register for the Paused state.
 3. Issue the Go command and poll for the Ready state.
- To take this arc under hardware control, use the AXI low-power interface to bring the DMC out of a low-power state.

Arc 9 to 6 The same as arc 8 to 6, but you must first start the **mclk** domain clock.

Arc 10 to 6 The same as arc 8 to 6, but you must first start the **acclk** domain clock.

Arc 11 to 6 The same as arc 8 to 6, but you must first start both the **acclk** and **mclk** domain clocks.

Arc 12 to 14 Apply power to the **acclk** power domain.

Arc 14 to 16 Assert reset to the **acclk** reset domain.

Arc 16 to 9 De-assert reset to the **acclk** reset domain.

Arc 13 to 15 Apply power to the **acclk** power domain.

Arc 15 to 17 Assert reset to the **acclk** reset domain.

Arc 17 to 8 De-assert reset to the **acclk** reset domain.

Chapter 3

Programmer's Model

This chapter describes the registers of the DMC (PL340) registers and provides information for programming the device. It contains the following sections:

- *About the programmer's model* on page 3-2
- *Register summary* on page 3-3
- *Register descriptions* on page 3-6.

3.1 About the programmer's model

Figure 3-1 shows the DMC register map.

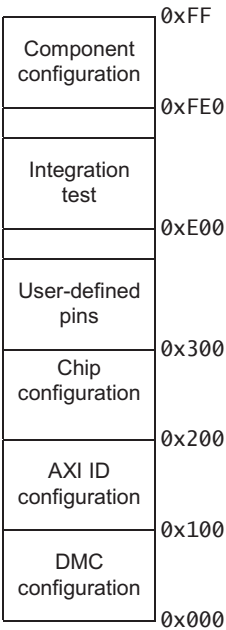


Figure 3-1 Register map

3.2 Register summary

Figure 3-2 shows the DMC configuration register map.

t_{ESR}	0x048
t_{XSR}	0x044
t_{XP}	0x040
t_{WTR}	0x03C
t_{WR}	0x038
t_{RRD}	0x034
t_{RP}	0x030
t_{RFC}	0x02C
t_{RCD}	0x028
t_{RC}	0x024
t_{RAS}	0x020
t_{MRD}	0x01C
t_{DQSS}	0x018
cas_latency	0x014
refresh_prd	0x010
memory_cfg	0x00C
direct_cmd	0x008
memc_cmd	0x004
memc_status	0x000

Figure 3-2 DMC configuration register map

Figure 3-3 on page 3-4 shows the AXI ID configuration registers map.

id_15_cfg	0x13C
⋮	
id_1_cfg	0x104
id_0_cfg	0x100

Figure 3-3 AXI ID configuration register map

Figure 3-4 shows the chip configuration register map.

chip_3_cfg	0x20C
chip_2_cfg	0x208
chip_1_cfg	0x204
chip_0_cfg	0x200

Figure 3-4 Chip configuration registers map

Figure 3-5 shows the component configuration register map.

comp_id_3	0xFFC
comp_id_2	0xFF8
comp_id_1	0xFF4
comp_id_0	0xFF0
periph_id_3	0xFEC
periph_id_2	0xFE8
periph_id_1	0xFE4
periph_id_0	0xFE0

Figure 3-5 Component configuration register map

Table 3-1 on page 3-5 lists the DMC registers.

Table 3-1 DMC register summary

Name	Base offset	Type	Reset value	Description
memc_status	0x000	RO	- ^a	<i>Memory Controller Status Register</i> on page 3-6
memc_cmd	0x004	WO	-	<i>Memory Controller Command Register</i> on page 3-7
direct_cmd	0x008	WO	-	<i>Direct Command Register</i> on page 3-8
memory_cfg	0x00C	RW	0x00010020	<i>Memory Configuration Register</i> on page 3-9
refresh_prd	0x010	RW	0x00000A60	<i>Refresh Period Register</i> on page 3-11
cas_latency	0x014	RW	0x00000006	<i>CAS Latency Register</i> on page 3-12
t_dqss	0x018	RW	0x00000001	<i>t_dqss Register</i> on page 3-12
t_mrd	0x01C	RW	0x00000002	<i>t_mrd Register</i> on page 3-13
t_ras	0x020	RW	0x00000007	<i>t_ras Register</i> on page 3-14
t_rc	0x024	RW	0x0000000B	<i>t_rc Register</i> on page 3-14
t_rcd	0x028	RW	0x0000001D	<i>t_rcd Register</i> on page 3-15
t_rfc	0x02C	RW	0x00000212	<i>t_rfc Register</i> on page 3-15
t_rp	0x030	RW	0x0000001D	<i>t_rp Register</i> on page 3-16
t_rrd	0x034	RW	0x00000002	<i>t_rrd Register</i> on page 3-16
t_wr	0x038	RW	0x00000003	<i>t_wr Register</i> on page 3-17
t_wtr	0x03C	RW	0x00000002	<i>t_wtr Register</i> on page 3-17
t_xp	0x040	RW	0x00000001	<i>t_xp Register</i> on page 3-18
t_xsr	0x044	RW	0x0000000A	<i>t_xsr Register</i> on page 3-18
t_esr	0x048	RW	0x00000014	<i>t_esr Register</i> on page 3-19
id_n_cfg	0x100	RW	0x00000000	<i>id_<n>_cfg Registers</i> on page 3-19
chip_n_cfg	0x200	RW	0x0000FF00	<i>chip_<n>_cfg Registers</i> on page 3-20
user_status	0x300	RO	-	<i>user_status Register</i> on page 3-21
user_config	0x304	WO	-	<i>user_config Register</i> on page 3-21
periph_id_n	0xFE0-0xFEC	RO	-	<i>Peripheral Identification Registers 0-3</i> on page 3-22
comp_id_n	0xFF0-0xFFC	RO	-	<i>Component Identification Registers 0-3</i> on page 3-24

a. Dependent on configuration

3.3 Register descriptions

This section describes the DMC registers.

3.3.1 Memory Controller Status Register

The read-only memc_status Register provides information on the configuration of the DMC and also the current state of the DMC. It cannot be read in either the Reset or POR states. Figure 3-6 shows the register bit assignments.

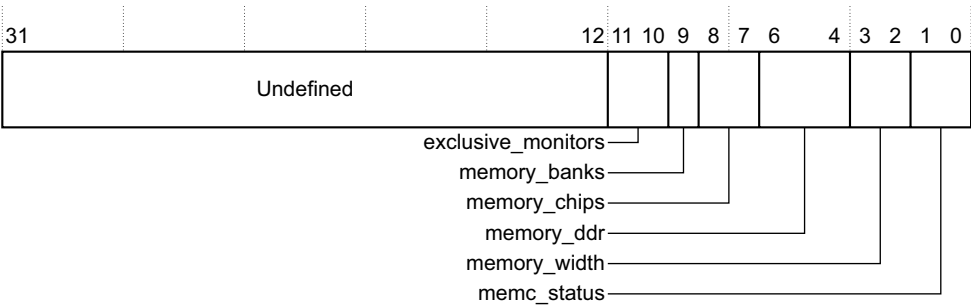


Figure 3-6 memc_status Register bit assignments

Table 3-2 lists the register bit assignments.

Table 3-2 memc_status Register bit assignments

Bits	Name	Function
[31:12]	-	Read undefined.
[11:10]	exclusive_monitors	Returns the number of exclusive access monitor resources implemented in the DMC: 2'b00 = 0 monitors 2'b01 = 1 monitor 2'b10 = 2 monitors 2'b11 = 4 monitors.
[9]	memory_banks	This returns the maximum number of banks that the PL340 supports on each chip. This is fixed at 1'b01 = 4 banks.
[8:7]	memory_chips	Returns the number of different chip selects that the DMC supports: 2'b00 = 1 chip 2'b01 = 2 chips 2'b10 = 3 chips 2'b11 = 4 chips.

Table 3-2 memc_status Register bit assignments (continued)

Bits	Name	Function
[6:4]	memory_ddr	Returns which SDRAM the DMC supports: 3'b000 = SDR SDRAM 3'b001 = DDR SDRAM 3'b011 = Mobile DDR SDRAM 3'b010 = Reserved 3'b1xx = Reserved. If Mobile DDR SDRAM or SDR SDRAM is supported the cas_half_cycle bit at address offset 0x14 is ignored.
[3:2]	memory_width	Returns the width of the external memory: 2'b00 = 16-bit 2'b01 = 32-bit 2'b10 = 64-bit 2'b11 = Reserved.
[1:0]	memc_status	Returns the state of the memory controller: 2'b00 = Config 2'b01 = Ready 2'b10 = Paused 2'b11 = Low_power.

3.3.2 Memory Controller Command Register

The write-only memc_cmd Register enables the DMC to be traversed. The command register controls the programmer's view FSM. By writing to this register the FSM can be traversed. If a new command is received to change state and a previous command to change state has not completed the **pready** signal is held LOW until the new command can be carried out.

Figure 3-7 shows the register bit assignments.

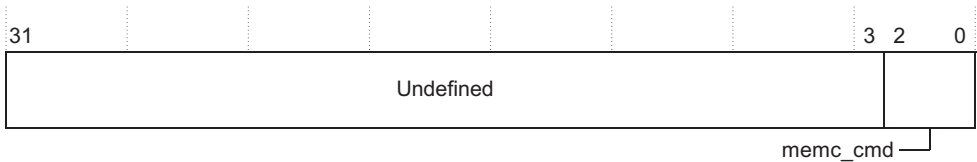


Figure 3-7 memc_cmd Register bit assignments

Table 3-3 lists the register bit assignments.

Table 3-3 memc_cmd Register bit assignments

Bits	Name	Function
[31:3]	-	Undefined. Write as zero.
[2:0]	memc_cmd	Changes the state of the memory controller: 3'b000 = Go 3'b001 = Sleep 3'b010 = Wakeup 3'b011 = Pause 3'b100 = Configure.

3.3.3 Direct Command Register

The write-only direct_cmd Register passes commands to the external memory. The configuration of the DirectCmd register enables you to write to any type of Mode register supported by the external memory device and also to generate NOP, Prechargeall, and Autorefresh commands. The DirectCmd Register therefore enables any initialization sequence that an external memory device might require. The only timing information associated with the DirectCmd Register are the command delays defined in the timing registers. Therefore if an initialization sequence requires further delays between commands they must be timed by the master driving the initialization sequence.

This register can only be written in the Config or Low-power state. Figure 3-8 shows the register bit assignments.

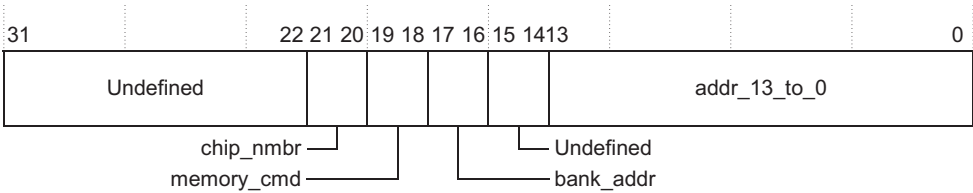


Figure 3-8 direct_cmd Register bit assignments

Table 3-4 lists the register bit assignments.

Table 3-4 direct_cmd Register bit assignments

Bits	Name	Function
[31:22]	-	Undefined. Write as zero.
[21:20]	chip_nmbr	Bits mapped to external memory chip address bits.
[19:18]	memory_cmd	Determines the command required: 2'b00 = Prechargeall 2'b01 = Autorefresh 2'b10 = Modereg or Extended modereg access 2'b11 = NOP.
[17:16]	bank_addr	Bits mapped to external memory bank address bits when command is Modereg access.
[15:14]	-	Undefined. Write as zero.
[13:0]	addr_13_to_0	Bits mapped to external memory address bits [13:0] when command is Modereg access.

3.3.4 Memory Configuration Register

The read/write memory_cfg Register configures the memory. It can only be read and written in the Config or Low-power state. Figure 3-9 shows the register bit assignments.

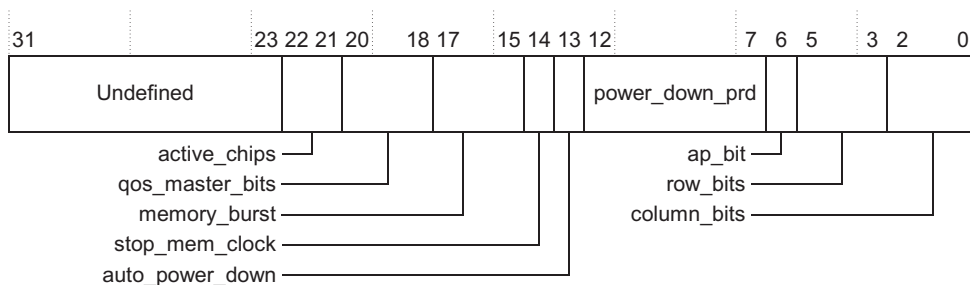


Figure 3-9 memory_cfg Register bit assignments

Table 3-5 lists the register bit assignments.

Table 3-5 memory_cfg Register bit assignments

Bits	Name	Function
[31:23]	-	Read undefined. Write as zero.
[22:21]	active_chips	Enables the refresh command generation for the number of memory chips. It is only possible to generate commands up to and including the number of chips in the configuration defined in the memc_status Register: 2'b00 = 1 chip 2'b01 = 2 chips 2'b10 = 3 chips 2'b11 = 4 chips.
[20:18]	qos_master_bits	Encodes the four bits of the 8-bit AXI ARID that are used to select one of the 16 QOS values: 3'b000 = ARID[3:0] 3'b001 = ARID[4:1] 3'b010 = ARID[5:2] 3'b011 = ARID[6:3] 3'b100 = ARID[7:4].
[17:15]	memory_burst	Encodes the number of data accesses that are performed to the SDRAM for each Read and Write command: 3'b000 = Burst 1 3'b001 = Burst 2 3'b010 = Burst 4 3'b011 = Burst 8 3'b100 = Burst 16. This value must also be programmed into the SDRAM mode register using the direct_cmd register at offset 0x8, and must match it.
[14]	stop_mem_clock	When enabled the memory clock is dynamically stopped when not performing an access to the SDRAM.
[13]	auto_power_down	When this is set, the memory interface automatically places the SDRAM into power-down state by deasserting CKE when the command FIFO has been empty for PowerDownPrd memory clock cycles.
[12:7]	power_down_prd	Number of memory clock cycles for auto power-down of the SDRAM.

Table 3-5 memory_cfg Register bit assignments (continued) (continued)

Bits	Name	Function
[6]	ap_bit	Encodes the position of the auto-precharge bit in the memory address: 1'b0 = address bit 10 1'b1 = address bit 8.
[5:3]	row_bits	Encodes the number of bits of the AXI address that comprise the row address: 3'b000 = 11 bits 3'b001 = 12 bits 3'b010 = 13 bits 3'b011 = 14 bits 3'b100 = 15 bits 3'b101 = 16 bits. The combination of row size, column size, BRC/RBC, and memory width must ensure that neither the MSB of the row address nor the MSB of the bank address exceeds address range [27:0]
[2:0]	column_bits	Encodes the number of bits of the AXI address that comprise the column address: 3'b000 = 8 bits 3'b001 = 9 bits 3'b010 = 10 bits 3'b011 = 11 bits 3'b100 = 12 bits.

3.3.5 Refresh Period Register

The read/write refresh_prd Register sets the memory refresh period. It can only be read and written in the Config or Low-power state. Figure 3-10 shows the register bit assignments.

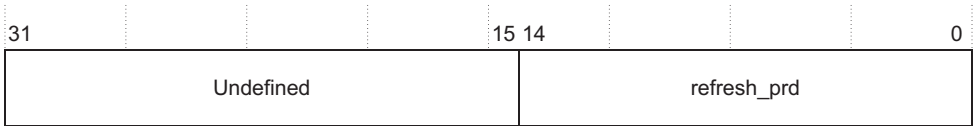


Figure 3-10 refresh_prd Register bit assignments

Table 3-6 lists the register bit assignments.

Table 3-6 refresh_prd Register bit assignments

Bits	Name	Function
[31:15]	-	Read undefined. Write as zero.
[14:0]	refresh_prd	Memory refresh period in memory clock cycles.

3.3.6 CAS Latency Register

The read/write cas_latency Register sets the CAS latency in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-11 shows the register bit assignments.

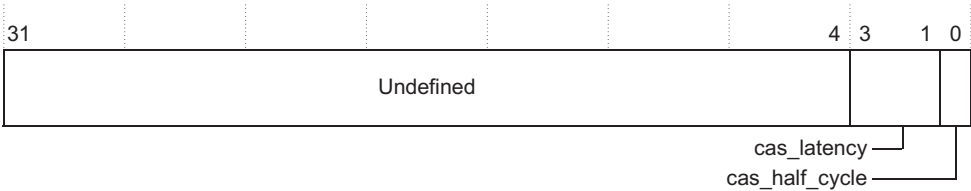


Figure 3-11 cas_latency Register bit assignments

Table 3-7 lists the register bit assignments.

Table 3-7 cas_latency Register bit assignments

Bits	Name	Function
[31:4]	-	Read undefined. Write as zero.
[3:1]	cas_latency	CAS latency in memory clock cycles.
[0]	cas_half_cycle	Encodes whether the CAS latency is half a memory clock cycle more than the value given in bits [3:1]: 1'b0 = Zero cycles offset to value in [3:1]. b0 is forced to 0 in MDDR and SDR mode. 1'b1 = Half cycle offset to value in [3:1].

3.3.7 t_dqss Register

The read/write t_dqss Register writes to DQS in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-12 on page 3-13 shows the register bit assignments.

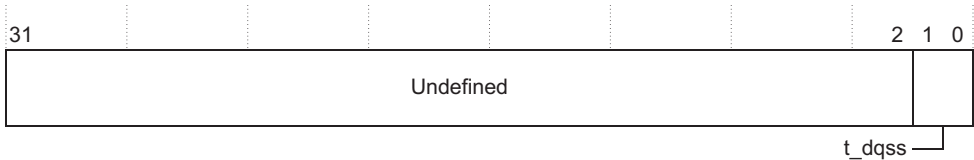


Figure 3-12 `t_dqss` Register bit assignments

Table 3-8 lists the register bit assignments.

Table 3-8 `t_dqss` Register bit assignments

Bits	Name	Function
[31:2]	-	Read undefined. Write as zero.
[1:0]	<code>t_dqss</code>	Write to DQS in memory clock cycles.

3.3.8 `t_mrd` Register

The read/write `t_mrd` Register sets the mode register command time in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-13 shows the register bit assignments.

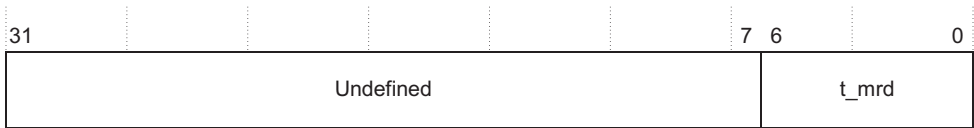


Figure 3-13 `t_mrd` Register bit assignments

Table 3-9 lists the register bit assignments.

Table 3-9 `t_mrd` Register bit assignments

Bits	Name	Function
[31:7]	-	Read undefined. Write as zero.
[6:0]	<code>t_mrd</code>	Set mode register command time in memory clock cycles.

3.3.9 t_ras Register

The read/write `t_ras` Register sets the RAS to precharge delay in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-14 shows the register bit assignments.

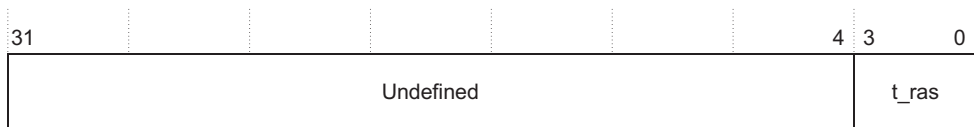


Figure 3-14 t_ras Register bit assignments

Table 3-10 lists the register bit assignments.

Table 3-10 t_ras Register bit assignments

Bits	Name	Function
[31:4]	-	Read undefined. Write as zero.
[3:0]	t_ras	Set RAS to precharge delay in memory clock cycles.

3.3.10 t_rc Register

The read/write `t_rc` Register sets the Active bank `x` to Active bank `x` delay in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-15 shows the register bit assignments.

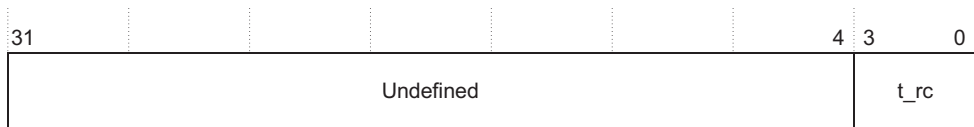


Figure 3-15 t_rc Register bit assignments

Table 3-11 lists the register bit assignments.

Table 3-11 t_rc Register bit assignments

Bits	Name	Function
[31:4]	-	Read undefined. Write as zero.
[3:0]	t_rc	Set Active bank x to Active bank x delay in memory clock cycles.

3.3.11 t_rcd Register

The read/write t_rcd Register sets the RAS to CAS minimum delay in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-16 shows the register bit assignments.

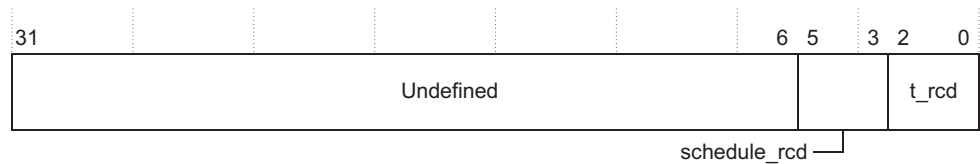


Figure 3-16 t_rcd Register bit assignments

Table 3-12 lists the register bit assignments.

Table 3-12 t_rcd Register bit assignments

Bits	Name	Function
[31:6]	-	Read undefined. Write as zero.
[5:3]	schedule_rcd	Set the RAS to CAS minimum delay in aclk cycles -3.
[2:0]	t_rcd	Set the RAS to CAS minimum delay in memory clock cycles.

3.3.12 t_rfc Register

The read/write t_rfc Register sets the auto-refresh command time in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-17 shows the register bit assignments.



Figure 3-17 t_rfc Register bit assignments

Table 3-13 lists the register bit assignments.

Table 3-13 t_rfc Register bit assignments

Bits	Name	Function
[31:10]	-	Read undefined. Write as zero.
[9:5]	schedule_rfc	Set the autorefresh command time in aclk cycles -3.
[4:0]	t_rfc	Set the auto-refresh command time in memory clock cycles.

3.3.13 t_rp Register

The read/write t_rp Register sets the precharge to RAS delay in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-18 shows the register bit assignments.

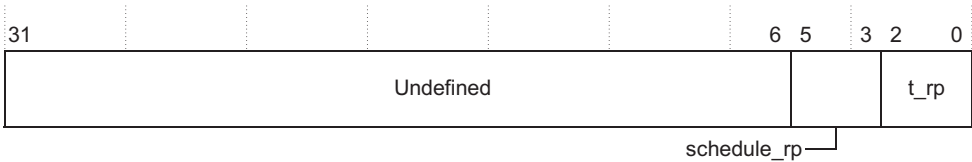


Figure 3-18 t_rp Register bit assignments

Table 3-14 lists the register bit assignments.

Table 3-14 t_rp Register bit assignments

Bits	Name	Function
[31:6]	-	Read undefined. Write as zero.
[5:3]	schedule_rp	Set the precharge to RAS delay in aclk cycles -3.
[2:0]	t_rp	Set the precharge to RAS delay in memory clock cycles.

3.3.14 t_rrd Register

The read/write t_rrd Register sets the Active bank x to Active bank y delay in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-19 on page 3-17 shows the register bit assignments.

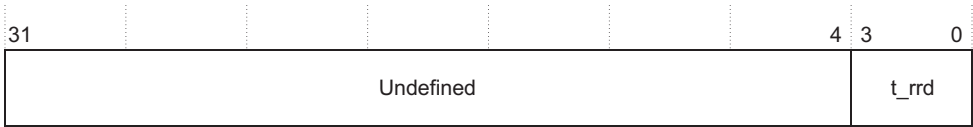


Figure 3-19 t_rrd Register bit assignments

Table 3-15 lists the register bit assignments.

Table 3-15 t_rrd Register bit assignments

Bits	Name	Function
[31:4]	-	Read undefined. Write as zero.
[3:0]	t_rrd	Set Active bank x to Active bank y delay in memory clock cycles.

3.3.15 t_wr Register

The read/write t_wr Register sets the write to precharge delay in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-21 on page 3-18 shows the register bit assignments.

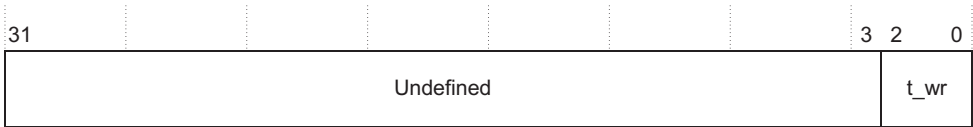


Figure 3-20 t_wr Register bit assignments

Table 3-17 on page 3-18 lists the register bit assignments.

Table 3-16 t_wr Register bit assignments

Bits	Name	Function
[31:3]	-	Read undefined. Write as zero.
[2:0]	t_wr	Set the write to precharge delay in memory clock cycles.

3.3.16 t_wtr Register

The read/write t_wtr Register sets the write to read delay in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-21 on page 3-18 shows the register bit assignments.

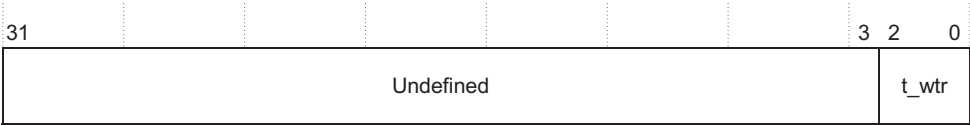


Figure 3-21 t_wtr Register bit assignments

Table 3-17 lists the register bit assignments.

Table 3-17 t_wtr Register bit assignments

Bits	Name	Function
[31:3]	-	Read undefined. Write as zero.
[2:0]	t_wtr	Set the write to read delay in memory clock cycles.

3.3.17 t_xp Register

The read/write t_xp Register sets exit power down command time in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-22 shows the register bit assignments.

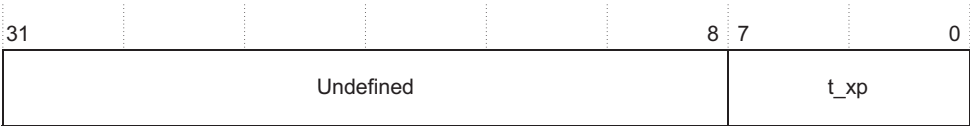


Figure 3-22 t_xp Register bit assignments

Table 3-18 lists the register bit assignments.

Table 3-18 t_xp Register bit assignments

Bits	Name	Function
[31:8]	-	Read undefined. Write as zero.
[7:0]	t_xp	Set the exit power down command time in memory clock cycles.

3.3.18 t_xsr Register

The read/write t_xsr Register sets exit self refresh command time in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-23 on page 3-19 shows the register bit assignments.

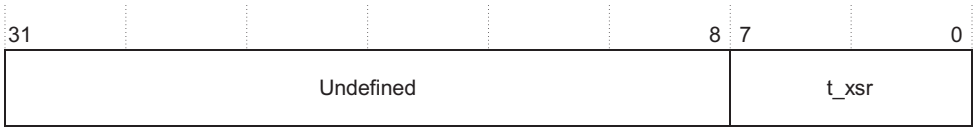


Figure 3-23 t_xsr Register bit assignments

Table 3-19 lists the register bit assignments.

Table 3-19 t_xsr Register bit assignments

Bits	Name	Function
[31:8]	-	Read undefined. Write as zero.
[7:0]	t_xsr	Set the exit self refresh command time in memory clock cycles.

3.3.19 t_esr Register

The read/write t_esr Register sets self refresh command time in memory clock cycles. It can only be read and written in the Config or Low-power state. Figure 3-24 shows the register bit assignments.

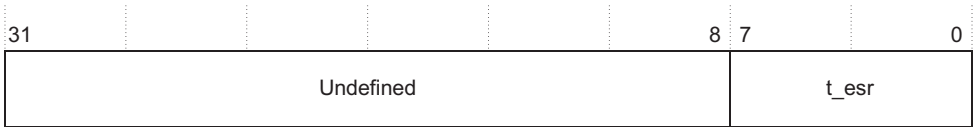


Figure 3-24 t_esr Register bit assignments

Table 3-20 lists the register bit assignments.

Table 3-20 t_esr Register bit assignments

Bits	Name	Function
[31:8]	-	Read undefined. Write as zero.
[7:0]	t_esr	Set the self refresh command time in memory clock cycles.

3.3.20 id_<n>_cfg Registers

The read/write id_<n>_cfg Registers are 16 registers that set the quality of service and span address locations 0x100-0x200. The registers can only be read and written in CONFIG state.

Figure 3-25 shows the register bit assignments.

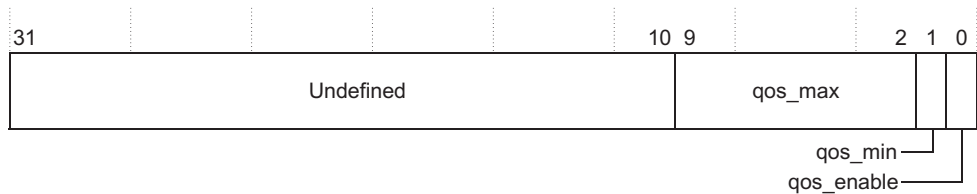


Figure 3-25 `id_<n>_cfg` Registers bit assignments

Table 3-21 lists the register bit assignments.

Table 3-21 `id_<n>_cfg` Registers bit assignments

Bits	Name	Function
[31:10]	-	Read undefined. Write as zero.
[9:2]	qos_max	Set a maximum quality of service.
[1]	qos_min	Set a minimum quality of service.
[0]	qoa_enable	Enables a quality of service value to be applied to memory reads from address ID n.

3.3.21 `chip_<n>_cfg` Registers

The read/write `chip_<n>_cfg` Registers are registers that set up the external memory device configuration. See *Release Note* for the number of external chips supported, and therefore the number of these registers. They span address locations `0x200-0x300`. There is one register per memory device. The registers configure the base address and address decoding method. The registers can only be read and written in CONFIG state.

Figure 3-26 shows the register bit assignments.

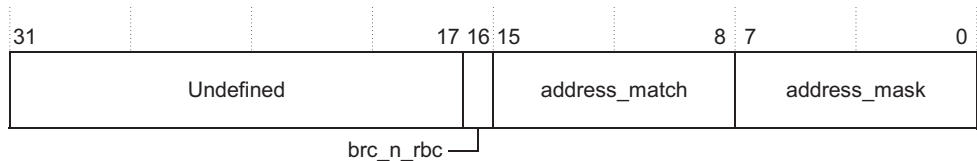


Figure 3-26 `chip_<n>_cfg` Registers bit assignments

Table 3-22 lists the register bit assignments.

Table 3-22 chip_<n>_cfg Registers bit assignments

Bits	Name	Function
[31:17]	-	Read undefined. Write as zero.
[16]	brc_n_rbc	Selects the memory organization as decoded from the AXI address: 1'b0 = Row, bank, column organization 1'b1 = Bank, row, column organization.
[15:8]	address_match	Comparison value for AXI address bits [31:24] to determine which chip is selected.
[7:0]	address_mask	The mask for AXI address bits [31:24] to determine which chip is selected: 1 = corresponding address bit is to be used for comparison.

3.3.22 user_status Register

This register returns the state of the **user_status[7:0]** primary inputs.

Table 3-23 lists the register bit assignments.

Table 3-23 user_status Registers bit assignments

Bits	Name	Function
[7:0]	user_status	Value of primary input pins

3.3.23 user_config Register

This register sets the value of the **user_config[7:0]** primary outputs.

Table 3-24 lists the register bit assignments.

Table 3-24 user_status Registers bit assignments

Bits	Name	Function
[7:0]	user_config	Sets primary output pins

3.3.24 Peripheral Identification Registers 0-3

The `periph_id` Registers are four eight-bit read-only registers, that span address locations `0xFE0-0xFEC`. The registers can conceptually be treated as a single register that holds a 32-bit peripheral ID value. They are read by an external master to determine which version of the device the DMC is.

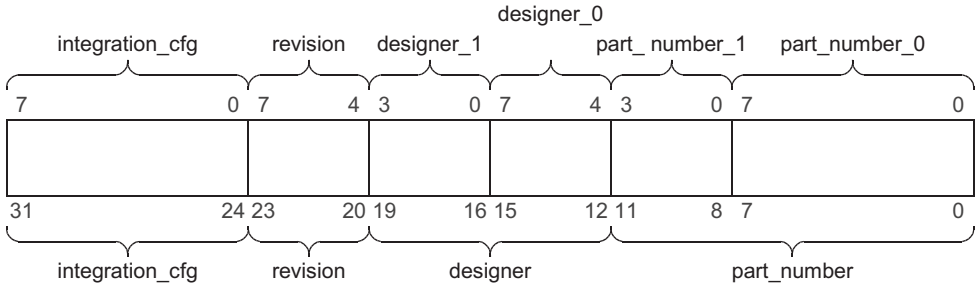
Table 3-25 lists the register bit assignments.

Table 3-25 `periph_id` Register bit assignments

Bits	Name	Description
[31:24]	<code>integration_cfg</code>	Configuration options are peripheral-specific. See <i>Peripheral Identification Register 3</i> on page 3-24.
[23:20]	-	The peripheral revision number is revision dependent.
[19:12]	<code>designer</code>	Designer's ID number. This is <code>0x41</code> for ARM.
[11:0]	<code>part_number</code>	Identifies the peripheral. The part number for PL340 is <code>0x340</code> .

Figure 3-27 shows the correspondence between bits of the `periph_id` registers and the conceptual 32-bit Peripheral ID Register.

Actual register bit assignment



Conceptual register bit assignment

Figure 3-27 `periph_id` Register bit assignments

The `periph_id` Registers are described in:

- *Peripheral Identification Register 0* on page 3-23
- *Peripheral Identification Register 1* on page 3-23
- *Peripheral Identification Register 2* on page 3-23

- *Peripheral Identification Register 3* on page 3-24.

Peripheral Identification Register 0

The `periph_id_0` Register is hard-coded and the fields within the register determine the reset value. Table 3-26 lists the register bit assignments.

Table 3-26 `periph_id_0` Register bit assignments

Bits	Name	Description
[31:8]	-	Read undefined
[7:0]	<code>part_number_0</code>	These bits read back as 0x40

Peripheral Identification Register 1

The `periph_id_1` Register is hard-coded and the fields within the register determine the reset value. Table 3-27 lists the register bit assignments.

Table 3-27 `periph_id_1` Register bit assignments

Bits	Name	Description
[31:8]	-	Read undefined
[7:4]	<code>designer_0</code>	These bits read back as 0x1
[3:0]	<code>part_number_1</code>	These bits read back as 0x3

Peripheral Identification Register 2

The `periph_id_2` Register is hard-coded and the fields within the register determine the reset value. Table 3-28 lists the register bit assignments.

Table 3-28 `periph_id_2` Register bit assignments

Bits	Name	Description
[31:8]	-	Read undefined.
[7:4]	<code>revision</code>	These bits read back as the revision number. This can be between 0 and 15. 0x0 is <code>r0p0</code> .
[3:0]	<code>designer_1</code>	These bits read back as 0x4.

Peripheral Identification Register 3

The `periph_id_3` register is hard-coded and the fields within the register determine the reset value. Table 3-29 lists the register bit assignments.

Table 3-29 `periph_id_3` Register bit assignments

Bits	Name	Description
[31:8]	-	Read undefined.
[7:4]	-	Reserved for future use. Read undefined.
[3:0]	Customer Modified	Customer modified number. 0 from ARM.

3.3.25 Component Identification Registers 0-3

The `comp_id` Registers are four eight-bit wide registers, that span address locations `0xFF0-0xFFC`. The registers can conceptually be treated as a single register that holds a 32-bit Component ID value. The register can be used for automatic BIOS configuration. The `comp_id` Register is set to `0xB105F00D`.

The register can be accessed with one wait state.

Figure 3-28 shows the register bit assignments.

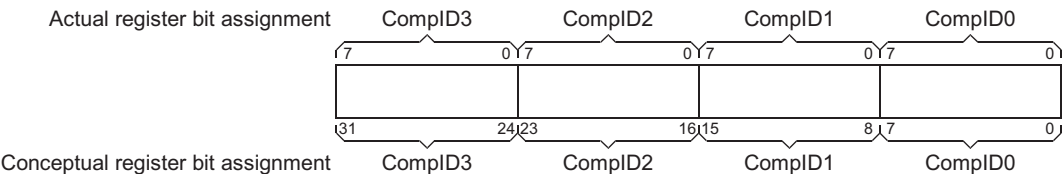


Figure 3-28 `comp_id` Register bit assignments

Table 3-30 lists the register bit assignments.

Table 3-30 comp_id Register bit assignments

Component ID register		CompID0-3 register		
Bits	Reset value	Register	Bits	Description
-	-	CompID3	[31:8]	Read undefined
[31:24]	0xB1	CompID3	[7:0]	These bits read back as 0xB1
-	-	CompID2	[31:8]	Read undefined
[23:16]	0x05	CompID2	[7:0]	These bits read back as 0x05
-	-	CompID1	[31:8]	Read undefined
[15:8]	0xF0	CompID1	[7:0]	These bits read back as 0xF0
-	-	CompID0	[31:8]	Read undefined
[7:0]	0x0D	CompID0	[7:0]	These bits read back as 0x0D

Chapter 4

Programmer's Model for Test

This chapter describes the additional logic for functional verification and production testing. It contains the following sections:

- *Integration test registers* on page 4-2.

4.1 Integration test registers

Test registers are provided for integration testing.

Figure 4-1 shows the Integration Test Register map.

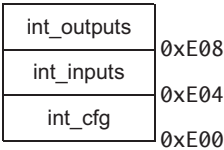


Figure 4-1 Integration Test Register map

Table 4-1 lists the integration test registers.

Table 4-1 DMC test register summary

Name	Base offset	Type	Reset value	Description
int_cfg	0xE00	RW	0x0	Integration Configuration Register
int_inputs	0xE04	RO	- ^a	Integration Inputs Register on page 4-3
int_outputs	0xE08	WO	-	Integration Outputs Register on page 4-4

a. Dependent on tie-off ports.

4.1.1 Integration Configuration Register

The read/write int_cfg Register selects the integration test registers. This register is only for test. It can only be read and written in Config state.

Figure 4-2 shows the register bit assignments.

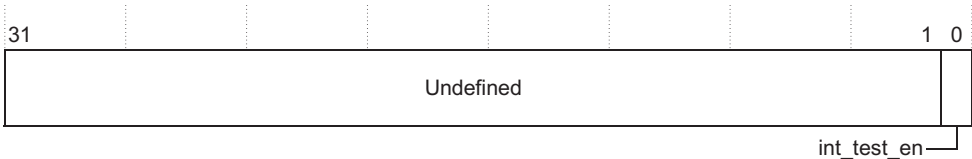


Figure 4-2 int_cfg Register bit assignments

Table 4-2 lists the register bit assignments.

Table 4-2 int_cfg Register bit assignments

Bits	Name	Function
[31:1]	Undefined	Read undefined. Write as zero.
[0]	int_test_en	When set, outputs are driven from the integration test registers, and input integration register values show external port states.

4.1.2 Integration Inputs Register

The read-only int_inputs Register enables an external master to access the inputs of the DMC using the APB interface. This register is only for test. It can only be read in Config state.

Figure 4-3 shows the register bit assignments.

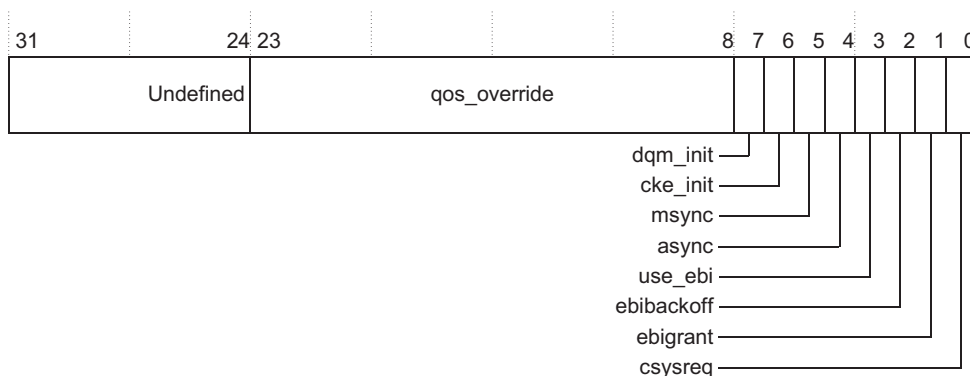


Figure 4-3 int_inputs Register bit assignments

Table 4-3 lists the register bit assignments.

Table 4-3 int_inputs Register bit assignments

Bits	Name	Function
[31:24]	-	Read undefined.
[23:8]	qos_override	Return the value of the qos_override external input
[7]	dqm_init	Returns the value of the dqm_init external input

Table 4-3 int_inputs Register bit assignments (continued)

Bits	Name	Function
[6]	cke_init	Returns the value of the cke_init external input
[5]	msync	Returns the value of the msync external input
[4]	async	Returns the value of the async external input
[3]	use_ebi	Returns the value of the use_ebi external input
[2]	ebibackoff	Returns the value of the ebibackoff external input
[1]	ebigrant	Returns the value of the ebigrant external input
[0]	csysreq	Returns the value of the csysreq external input

4.1.3 Integration Outputs Register

The write-only int_outputs Register enables an external master to access the outputs of the DMC using the APB interface. It can only be read in Config state.

Figure 4-4 shows the register bit assignments.

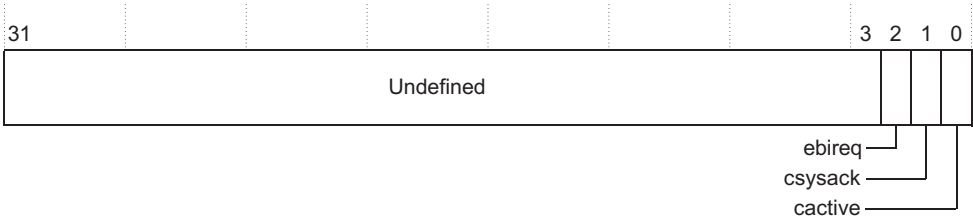


Figure 4-4 int_outputs Register bit assignments

Table 4-4 lists the register bit assignments.

Table 4-4 int_outputs Register bit assignments

Bits	Name	Function
[31:3]	-	Undefined. Write as zero.
[2]	ebireq	Drives the ebireq external output.
[1]	csysack	Drives the csysack external output.
[0]	cactive	Drives the cactive external output.

Chapter 5

Device Driver

This chapter describes the device driver. It contains the following section:

- *Device driver functions* on page 5-2.

5.1 Device driver functions

The DMC driver contains the set of functions that enable initialization of the memory controller, initialization of the memory devices, control of the memory controller state, and reading of the status of the memory controller.

5.1.1 Register access functions

These functions are defined in the header `ap1340.h` and source code is contained in `p1340.c`.

apPL340_StatusGet

This function reads the status bits of the Memory Controller Status Register to determine the current state of the DMC.

apPL340_FixedConfigGet

This function reads the hardware configuration from the Memory Controller Status Register.

apPL340_CommandSet

This function writes commands to the Memory Controller Command Register which change the state of the DMC.

apPL340_DirectCommandSet

This function writes data to the Direct Command Register which generates commands to external memory.

apPL340_MemConfigSet

This function writes memory configuration data to the Memory Controller Configuration Register which define the structure and features of the memory.

apPL340_MemConfigGet

This function is the complement of `apPL340_MemConfigSet`. to read the configuration data from the Memory Controller Configuration Register and format it in the same way as that passed to `apPL340_MemConfigSet`.

apPL340_RefreshPeriodSet

This function sets the refresh period to the Refresh Period Register.

apPL340_RefreshPeriodGet

This function is the complement of apPL340_RefreshPeriodSet to read the refresh period from the memory controller Refresh Period Register.

apPL340_TimingConfigSet

This function writes timing configuration data to the DMC Timing Registers.

apPL340_TimingConfigGet

This function is the complement of apPL340_TimingConfigSet to read timing data from DMC Timing Registers and format it in the same way as that passed to apPL340_TimingConfigSet.

apPL340_IdConfigSet

This function writes quality of service data to the id_<n>_cfg Register.

apPL340_IdConfigGet

This function is the complement of apPL340_IdConfigSet to read quality of service data from the id_<n>_cfg Register.

apPL340_ChipConfigSet

This function writes the external memory device configuration to the chip_<n>_cfg Registers of the memory controller.

apPL340_ChipConfigGet

This function is the complement of apPL340_ChipConfigSet to read the configuration of external memory from the chip_<n>_cfg Registers in the memory controller.

apPL340_UserStatusGet

This function reads the user-defined user_status Register.

apPL340_UserConfigSet

This function writes to the user-defined user_config Register.

apPL340_Initialize

This function is part of the PrimeCell driver framework and initializes the memory controller driver.

apPL340_StateSizeGet

This function is part of the PrimeCell driver framework and returns the memory required for the state structure in the PrimeCell driver.

5.1.2 Example memory initialization functions

These functions show typical initialization sequences for the memory and are contained in appl340tst.c. These functions implement the example setup shown in Table 2-1 on page 2-25.

apPL340_Initialization

This function initializes the memory controller with memory-specific data that is passed to the register access functions. This sequence of function calls sets up:

1. The timing parameters.
2. Memory configuration.
3. Chip selects for the memory system.

apPL340_InitializeMemory

This function initializes the memory mode registers by calling the apPL340_DirectCommandSet function multiple times, performing a memory-specific sequence of commands to set up the memory.

This function must only be called after apPL340_Initialization because that function sets up the mapping of the Direct Command Register to the physical addressing of external memory.

Appendix A

Signal Descriptions

This chapter describes the AMBA AXI, AMBA APB, and module-specific non-AMBA signals used with the DMC. It contains the following sections:

- *Clock and reset signals* on page A-2
- *Miscellaneous signals* on page A-3
- *AXI signals* on page A-4
- *APB signals* on page A-9
- *Pad interface signals* on page A-10.

A.1 Clock and reset signals

Table A-1 lists the clock and reset signals.

Table A-1 Clock and reset signals

Signal	Type	Source/ destination	Description
aclk	Input	Clock source	Clock for the aclk domain.
aresetn	Input	Reset source	aclk domain reset signal. This signal is active LOW.
mclk	Input	Clock source	Clock for mclk domain.
mclkn	Input	Clock source	Optional clock for pad interface block.
mclcx2	Input	Clock source	Optional clock for pad interface block.
mclcx2n	Input	Clock source	Optional clock for pad interface block.
mresetn	Input	Reset source	Reset for mclk domain. This signal is active LOW.

A.2 Miscellaneous signals

Table A-9 on page A-9 lists the miscellaneous signals.

Table A-2 Miscellaneous signals

Signal	Type	Source	Description
a_gt_m_sync	Input	Tie-off pin	When HIGH indicates aclk is greater than mclk but is still synchronous.
async	Input	Tie-off pin	When HIGH indicates aclk is synchronous to mclk . Otherwise they are asynchronous. async must be tied to the same value as msync .
cke_init	Input	Tie-off pin	The cke output port to the external memory resets to this value.
dft_en_clk_out	Input	Tie-off pin	This signal is used for ATPG testing only.
dqm_init	Input	Tie-off pin	The dqm output ports to the external memory reset to this value.
memory_width[1:0]	Input	Tie-off pin	These configure the external memory width. See Table 3-2 on page 3-6 for the definition of these two pins. See the Release Note for legal values of memory width for each configuration.
msync	Input	Tie-off pin	When HIGH indicates mclk is synchronous to aclk . Otherwise they are asynchronous. msync must be tied to the same value as async .
qos_override[15:0]	Input	External control logic	When one or more bits are HIGH, coincident with arvalid and arready , and when the arid match bits are equivalent to the qos_override bit(s), then the quality of service for the read access is forced to minimum latency.
rst_bypass	Input	Tie-off pin	This signal is used for ATPG testing only.
use_ebi	Input	Tie-off pin	When HIGH indicates that the DMC must operate with a PrimeCell EBI (PL220).
user_config[7:0]	Output	External control logic	General purpose APB accessible control pins. For example this could be used for DLL control.
user_status[7:0]	Input	External control logic	General-purpose APB accessible input pin.

A.3 AXI signals

The AXI signals are described in the following sections:

- *Write address channel signals*
- *Write data channel signals* on page A-5
- *Buffered write response channel signals* on page A-6
- *Read address channel signals* on page A-6
- *Read data channel signals* on page A-7
- *AXI low-power interface signals* on page A-8.

A.3.1 Write address channel signals

Table A-3 lists the AXI write address channel signals.

Table A-3 Write address channel signals

Signal	Type	Source/ destination	Description
awid[7:0]	Input	Master	Address ID. This signal is the ID tag of the read or write transaction.
awaddr[31:0]	Input	Master	Address. The address bus gives the address of the first transfer in a burst. The associated control signals are used to determine the addresses of the remaining transfers in the burst.
awlen[3:0]	Input	Master	Burst length. This signal indicates the number of transfers in a burst.
awsize[2:0]	Input	Master	Burst size. This signal indicates the size of each transfer in a burst. For write transfers, byte lane strobes indicate which byte lanes to update in memory.
awburst[1:0]	Input	Master	Burst type. This signal indicates a fixed, incrementing, or wrapping burst. The AXI uses awburst[1:0] and awsize[2:0] to calculate the address for successive transfers in the burst.
awlock[1:0]	Input	Master	Lock type. This signal indicates a normal, exclusive, or locked transaction.
awcache[3:0]	Input	Master	Cache type. This signal indicates the bufferable, cacheable, write-through, write-back, and allocate attributes of the transaction.

Table A-3 Write address channel signals (continued)

Signal	Type	Source/ destination	Description
awprot[1:0]	Input	Master	Protection level. This signal indicates the normal, privileged, or secure protection level of the transaction and whether the transaction is a data access or an instruction access.
awvalid	Input	Master	Address valid. This signal indicates that valid address and control information are available: 1 = address and control information available 0 = address and control information not available. The address and control information remain stable until the address acknowledge signal, aready , goes HIGH.
awready	Output	Slave	Address ready. This signal indicates that the slave is ready to accept an address: 1 = slave ready 0 = slave not ready.

A.3.2 Write data channel signals

Table A-4 lists the AXI write data channel signals.

Table A-4 Write data channel signals

Signal	Type	Source/ destination	Description
wid[7:0]	Input	Master	Write ID tag. This signal is the ID tag of the write transfer. The wid value must match the awid value of the write transaction.
wdata[PORTWIDTH-1:0]^a	Input	Master	Write data. The write data bus can be 32, 64 or 128 bits wide.
wstrb[PORTBYTES-1:0]^b	Input	Master	Write strobes. This signal indicates which byte lanes to update in memory. There is one write strobe for each eight bits of the write data bus. Therefore, wstrb[n] corresponds to wdata[(8 × n) + 7:(8 × n)] .
wlast	Input	Master	Write last. This signal indicates the last transfer in a write burst.
wvalid	Input	Master	Write valid. This signal indicates that valid write data and strobes are available: 1 = write data and strobes available 0 = write data and strobes not available.
wready	Output	Slave	Write ready. This signal indicates that the slave can accept the write data: 1 = slave ready 0 = slave not ready.

a. PORTWIDTH is the data width of the AXI port in bits.

b. PORTBYTES is the data width of the AXI port in bytes.

A.3.3 Buffered write response channel signals

Table A-5 lists the buffered write response channel signals.

Table A-5 Buffered write response channel signals

Signal	Type	Source/ destination	Description
bid[7:0]	Output	Slave	Response ID. The identification tag of the write response. The bid value must match the awid value of the write transaction to which the slave is responding.
bresp[1:0]	Output	Slave	Write response. This signal indicates the status of the write transaction. The allowable responses are OKAY and EXOKAY.
bvalid	Output	Slave	Write response valid. This signal indicates that a valid write response is available: 1 = write response available 0 = write response not available.
bready	Input	Master	Response ready. This signal indicates that the master can accept the response information. 1 = master ready 0 = master not ready.

A.3.4 Read address channel signals

Table A-6 lists the AXI read address channel signals.

Table A-6 Read address channel signals

Signal	Type	Source/ destination	Description
arid[7:0]	Input	Master	Address ID. This signal is the ID tag of the read or write transaction.
araddr[31:0]	Input	Master	Address. The address bus gives the address of the first transfer in a burst. The associated control signals are used to determine the addresses of the remaining transfers in the burst.
arlen[3:0]	Input	Master	Burst length. This signal indicates the number of transfers in a burst.
arsize[2:0]	Input	Master	Burst size. This signal indicates the size of each transfer in a burst. For write transfers, byte lane strobes indicate which byte lanes to update in memory.
arburst[1:0]	Input	Master	Burst type. This signal indicates a fixed, incrementing, or wrapping burst. The AXI uses aburst[1:0] and asize[2:0] to calculate the address for successive transfers in the burst.
arlock[1:0]	Input	Master	Lock type. This signal indicates a normal, exclusive, or locked transaction.
arcache[3:0]	Input	Master	Cache type. This signal indicates the bufferable, cacheable, write-through, write-back, and allocate attributes of the transaction.

Table A-6 Read address channel signals (continued)

Signal	Type	Source/ destination	Description
arprot[1:0]	Input	Master	Protection level. This signal indicates the normal, privileged, or secure protection level of the transaction and whether the transaction is a data access or an instruction access.
arvalid	Input	Master	Address valid. This signal indicates that valid address and control information are available: 1 = address and control information available 0 = address and control information not available. The address and control information remain stable until the address acknowledge signal, aready , goes HIGH.
aready	Output	Slave	Address ready. This signal indicates that the slave is ready to accept an address: 1 = slave ready 0 = slave not ready.

A.3.5 Read data channel signals

Table A-7 lists the AXI read data channel signals.

Table A-7 Read data channel signals

Signal	Type	Source/ destination	Description
rid[7:0]	Output	Slave	Read ID tag. This signal is the ID tag of the read transfer. The rid value must match the arid value of the read transaction to which the slave is responding.
rdata[PORTWIDTH-1:0]^a	Output	Slave	Read data. The read data bus can be 32, 64, or 128 bits wide.
rresp[1:0]	Output	Slave	Read response. This signal indicates the status of the read transfer. The allowable responses are OKAY and EXOKAY.
rlast	Output	Slave	Read last. This signal indicates the last transfer in a read burst.
rready	Input	Master	Read ready. This signal indicates that the master can accept the read data and response information: 1 = master ready 0 = master not ready.
rvalid	Output	Slave	Read valid. This signal indicates that valid read data is available: 1 = read data available 0 = read data not available.

a. PORTWIDTH is the data width of the AXI port in bits.

A.3.6 AXI low-power interface signals

Table A-8 lists the optional low-power interface signals.

Table A-8 AXI low-power interface signals

Signal	Type	Source/ destination	Description
cclken	Input	Bus clock	Clock enable.
csysreq	Input	Clock controller	System low-power request. This signal is a request from the system clock controller for the peripheral to enter a low-power state.
csysack	Output	Peripheral device	Low-power request acknowledgement. This signal is the acknowledgement from a peripheral of a system low-power request.
cactive	Output	Peripheral device	Clock active. This signal indicates that the peripheral requires its clock signal: 1 = peripheral clock required 0 = peripheral clock not required.

A.4 APB signals

Table A-9 lists the APB signals.

Table A-9 APB signals

Signal	Type	Source/ destination	Description
paddr[11:0]	Input	APB address bus	This is the APB address bus, which may be up to 32-bits wide and is driven by the peripheral bus bridge unit.
pclken	Input	Bus clock	Clock enable for clk in APB domain.
penable	Input	APB strobe	This strobe signal is used to time all accesses on the peripheral bus. The enable signal is used to indicate the second cycle of an APB transfer. The rising edge of penable occurs in the middle of the APB transfer.
prdata[31:0]	Output	APB read data bus	The read data bus is driven by the selected slave during read cycles (when pwrite is LOW). The read data bus can be up to 32-bits wide.
pready	Output	APB	APB transfer wait signal.
psel	Input	APB select	A signal from the secondary decoder, within the peripheral bus bridge unit, to each peripheral bus slave x. This signal indicates that the slave device is selected and a data transfer is required. There is a psel signal for each bus slave.
pslverr	Output	APB transfer error	This is tied off within the DMC. Included for completeness.
pdata[31:0]	Input	APB write data bus	The write data bus is driven by the peripheral bus bridge unit during write cycles (when pwrite is HIGH). The write data bus can be up to 32-bits wide.
pwrite	Input	APB transfer direction	When HIGH this signal indicates an APB write access and when LOW a read access.

A.5 Pad interface signals

Table A-10 lists the APB signal names.

Table A-10 Pad interface signals

Signal	Type	Source/ destination	Description
addr[15:0]	Output	External memory	Address bus
ap	Output	External memory	Auto precharge bit
ba[1:0]	Output	External memory	Bank select
cas_n	Output	External memory	Column address strobe
cke	Output	External memory	Clock enable
clk_out[MEMORIES-1:0]^a	Output	External memory	Memory clock
cs_n[MEMORIES-1:0]	Output	External memory	Chip select
data_en	Output	External memory	Data direction enable
dqm[MEMBYTES-1:0]^b	Output	External memory	Data bus mask bits
dqs_in_<n>	Input	External memory	Data strobe in, DDR only
dq_in[MEMWIDTH-1:0]^c	Input	External memory	Data bus input
dqs_in_n_<n>	Input	External memory	Data strober bar, DDR only
dqs_out_<n>	Output	External memory	Data strobe out, DDR only
dq_out[MEMWIDTH-1:0]	Output	External memory	Data bus output
fbclk_in	Input	External memory	Feedback clock
ras_n	Output	External memory	Row address strobe
we_n	Output	External memory	Write enable

a. MEMORIES is the number of chip selects.

b. MEMBYTES is the data width of the external memory bus in bytes.

c. MEMWIDTH is the data width of the external memory bus in bits.

Glossary

This glossary describes some of the terms used in this manual. Where terms can have several meanings, the meaning presented here is intended.

Abort

A mechanism that indicates to a core that it must halt execution of an attempted illegal memory access. An abort can be caused by the external or internal memory system as a result of attempting to access invalid instruction or data memory. An abort is classified as a Prefetch Abort, a Data Abort, or an External Abort.

See also Data Abort, External Abort, and Prefetch Abort.

Advanced eXtensible Interface (AXI)

This is a bus protocol that supports separate address/control and data phases, unaligned data transfers using byte strobes, burst-based transactions with only start address issued, separate read and write data channels to enable low-cost DMA, ability to issue multiple outstanding addresses, out-of-order transaction completion, and easy addition of register stages to provide timing closure. The AXI protocol also includes optional extensions to cover signaling for low-power operation.

AXI is targeted at high performance, high clock frequency system designs and includes a number of features that make it very suitable for high speed sub-micron interconnect.

Advanced High-performance Bus (AHB)

The AMBA Advanced High-performance Bus system connects embedded processors such as an ARM core to high-performance peripherals, DMA controllers, on-chip memory, and interfaces. It is a high-speed, high-bandwidth bus that supports multi-master bus management to maximize system performance.

See also Advanced Microcontroller Bus Architecture and AHB-Lite.

Advanced Microcontroller Bus Architecture (AMBA)

AMBA is the ARM open standard for multi-master on-chip buses, capable of running with multiple masters and slaves. It is an on-chip bus specification that describes a strategy for the interconnection and management of functional blocks that make up a *System-on-Chip* (SoC). It aids in the development of embedded processors with one or more CPUs or signal processors and multiple peripherals. AMBA complements a reusable design methodology by defining a common backbone for SoC modules. AHB conforms to this standard.

See also Advanced High-performance Bus and AHB-Lite.

Advanced Peripheral Bus (APB)

The AMBA Advanced Peripheral Bus is a simpler bus protocol than AHB. It is designed for use with ancillary or general-purpose peripherals such as timers, interrupt controllers, UARTs, and I/O ports. Connection to the main system bus is through a system-to-peripheral bus bridge that helps to reduce system power consumption.

See also Advanced High-performance Bus.

AHB

See Advanced High-performance Bus.

Aligned

Refers to data items stored so that their address is divisible by the highest power of two that divides their size. Aligned words and halfwords therefore have addresses that are divisible by four and two respectively. The terms word-aligned and halfword-aligned therefore refer to addresses that are divisible by four and two respectively. The terms byte-aligned and doubleword-aligned are defined similarly.

AMBA

See Advanced Microcontroller Bus Architecture.

APB

See Advanced Peripheral Bus.

Architecture

The organization of hardware and/or software that characterizes a processor and its attached components, and enables devices with similar characteristics to be grouped together when describing their behavior, for example, Harvard architecture, instruction set architecture, ARMv6 architecture.

AXI

See Advanced eXtensible Interface.

Big-endian	Memory organization in which the least significant byte of a word is at a higher address than the most significant byte. <i>See also</i> Little-endian and Endianness.
Central Processing Unit (CPU)	The part of a processor that contains the ALU, the registers, and the instruction decode logic and control circuitry. Also commonly known as the processor core.
Clock gating	Gating a clock signal for a macrocell with a control signal, and using the modified clock that results to control the operating state of the macrocell.
Coprocessor	A processor that supplements the main CPU. It carries out additional functions that the main CPU cannot perform. Usually used for floating-point math calculations, signal processing, or memory management.
CPU	<i>See</i> Central Processing Unit.
Doubleword	A 64-bit data item. The contents are taken as being an unsigned integer unless otherwise stated.
Endianness	Byte ordering. The scheme that determines the order in which successive bytes of a data word are stored in memory. <i>See also</i> Little-endian and Big-endian.
Little-endian	Memory organization where the least significant byte of a word is at a lower address than the most significant byte. <i>See also</i> Big-endian and Endianness.
Processor	A contraction of microprocessor. A processor includes the CPU or core, plus additional components such as memory, and interfaces. These are combined as a single macrocell, that can be fabricated on an integrated circuit.
Register	A temporary storage location used to hold binary data until it is ready to be used.
Reserved	A field in a control register or instruction format is reserved if the field is to be defined by the implementation, or produces Unpredictable results if the contents of the field are not zero. These fields are reserved for use in future extensions of the architecture or are implementation-specific. All reserved bits not used by the implementation must be written as zero and are read as zero.
Unaligned	Memory accesses that are not appropriately word-aligned or halfword-aligned. <i>See also</i> Aligned.

Unpredictable

For reads, the data returned when reading from this location is unpredictable. It can have any value. For writes, writing to this location causes unpredictable behavior, or an unpredictable change in device configuration. Unpredictable instructions must not halt or hang the processor, or any part of the system.